CoworkOS: Scaling Agent Coworkers for Real-world Automation

CoworkOS Team

Abstract

Rapid advances in cowork agents are shifting human-AI collaboration toward real-world automation, delivering software, designs, and other artifacts. However, such productivity work demands both high-quality delivery on complex productivity tasks and continued monitoring and decision-making over long horizons, a combination that remains an open challenge. We introduce CoworkOS, a general-purpose harness for long-horizon work that separates task strategy from execution management: the model decides actions, delegation, and acceptance, while the runtime maintains state and feedback. Its three-layer architecture enables direct execution or delegation to Subagents, synchronously or asynchronously, under stable responsibilities. Through a shared application layer, humans and agents inspect and revise the same evolving documents, presentations, CAD models, and Blender scenes, so human-in-the-loop participation can enter both through conversation and through direct modification of the work itself. To sustain continuity across agents and sessions, CoworkOS further introduces hierarchical workspaces and persistent semantic memory, so working files, key decisions, and long-running Job logs remain shareable, reusable, and resumable after interruption. Across 12 benchmarks and 6 categories of real-world applications, CoworkOS proves highly competitive with frontier agent applications such as Codex and DeepSeek Harness. As a sustained-work case study, CoworkOS uses GLM-5.3 as its backbone to conduct RSI-driven automated post-training on Qwen3.6-35B-A3B. Over five rounds, the model's average score across eight benchmarks rose from 33.19 to 42.95 (+9.76). Looking ahead, we believe CoworkOS opens a meaningful path toward general cowork-agent intelligence.

Project Page: coworkos-ai.github.io
Repo Link: github.com/RUC-NLPIR/CoworkOS
Demo Gallery: coworkos-ai.github.io/showcase.html

Contact: jinjiajie@ruc.edu.cn, tgui@fudan.edu.cn, dou@ruc.edu.cn

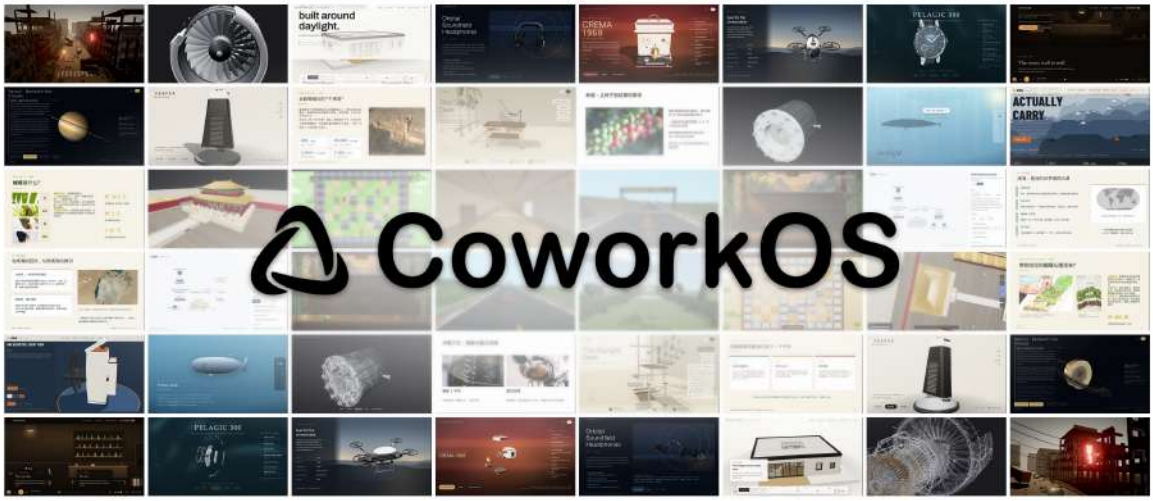


Figure 1: Overview of 24 application cases produced with CoworkOS across six categories: frontend websites, games, Office presentations, interactive visualizations, CAD, and Blender rendering. Two views of each case illustrate different pages, scenes, or inspection states. The montage provides a visual overview; harness comparisons and detailed inspection evidence follow in Figure 6 and Appendix A.2.

1 Introduction

Agent intelligence is advancing toward lifelong autonomy systems that continually interact and evolve with real-world environments. Progress toward Artificial Super Intelligence (ASI) requires attention to both model capability and the runtime systems that translate reasoning into action. Ideally, such agents would invoke tools, interpret feedback, maintain working state, and continue when their immediate interaction ends. To achieve this, the *agent harness* supplies the runtime mechanisms for this process, including tool orchestration, state and context management, and execution feedback. Because the harness shapes how a model acts and how well it can use available resources, models and their harnesses need to co-evolve (Liu et al., 2024; Sumers et al., 2024).

Recently, the evolution of agent systems can be understood through three progressive stages. **Agent Workflow** centers on reasoning and tool-using trajectories: early mechanisms include Chain-of-Thought and Tree-of-Thoughts (Wei et al., 2022; Yao et al., 2023a), interleaved reasoning and action in ReAct (Yao et al., 2023b), and feedback-driven reflection in Reflexion (Shinn et al., 2023). Workflow frameworks and construction methods, including DSPy (Khattab et al., 2023), AFlow (Zhang et al., 2024), and MetaGPT (Hong et al., 2024), further organize these capabilities. **Agent Assistant** extends this scope to persistent individual needs, bringing memory, messaging, code execution, and tool interfaces into continuing interactions. Frameworks such as LangChain, LangGraph (LangChain, 2026), AutoGen (Wu et al., 2023), and OpenClaw support different aspects of this development. The emerging **Agent Cowork** setting places agents within professional work: people and agents contribute to evolving artifacts, adjust their plans as work develops, and share initiative in complex productivity tasks. Coding and work-environment agents represented by Claude Code, Codex, OpenHands, and DeepSeek Harness have demonstrated strong real-world productivity (Anthropic, 2026; DeepSeek AI, 2026; Jimenez et al., 2024; OpenAI, 2026; Wang et al., 2025), while cowork agents such as Kimi Code, ZCode, Qoder, and Hermes Agent have also been widely adopted. Notably, these stages describe an expanding design lens: from automation by the agent itself, gradually toward sustained interactive work involving humans, agents, and the environment (Liu et al., 2024; Xie et al., 2024).

Building on this insight, we argue that this expansion of productivity reveals two complementary requirements for today's cowork agents, illustrated in Figure 2(a):

- **Complex productivity tasks:** producing software, documents, designs, and other artifacts through demanding reasoning, tool use, and verification. Such tasks call for the agent's comprehensive agentic intelligence and, where the work requires it, collective intelligence across collaborating agents, to carry a difficult task through to completion.
- **Continued monitoring and decision-making:** monitoring a training process over days, following a project across sessions, or watching external information and deciding whether and how to act when a relevant event occurs. Such monitoring requires the agent to keep its context healthy over time, maintain sound memory, and continuously manage and track the artifacts it produces.

Notably, both settings require sustained collaboration between humans and agents, which we treat as a basic design principle for cowork agents: the two sides work together within a shared application ecosystem centered on evolving work (Figure 2(b)). This leads to our central question: can a single cowork agent handle both complex productivity tasks and continued monitoring and decision-making, while keeping humans in the loop?

We propose CoworkOS, a cowork harness for long-horizon and human-in-the-loop task execution. To achieve this, a natural approach is a division of responsibility: *the model autonomously chooses the task-execution strategy, while the runtime harness maintains efficient, long-horizon cowork mechanisms*. Specifically, CoworkOS consists of a **three-layer agent framework** comprising a Main Agent, a Coordinator Runtime, and optional Subagents. Main sets the overall task plan and execution direction,

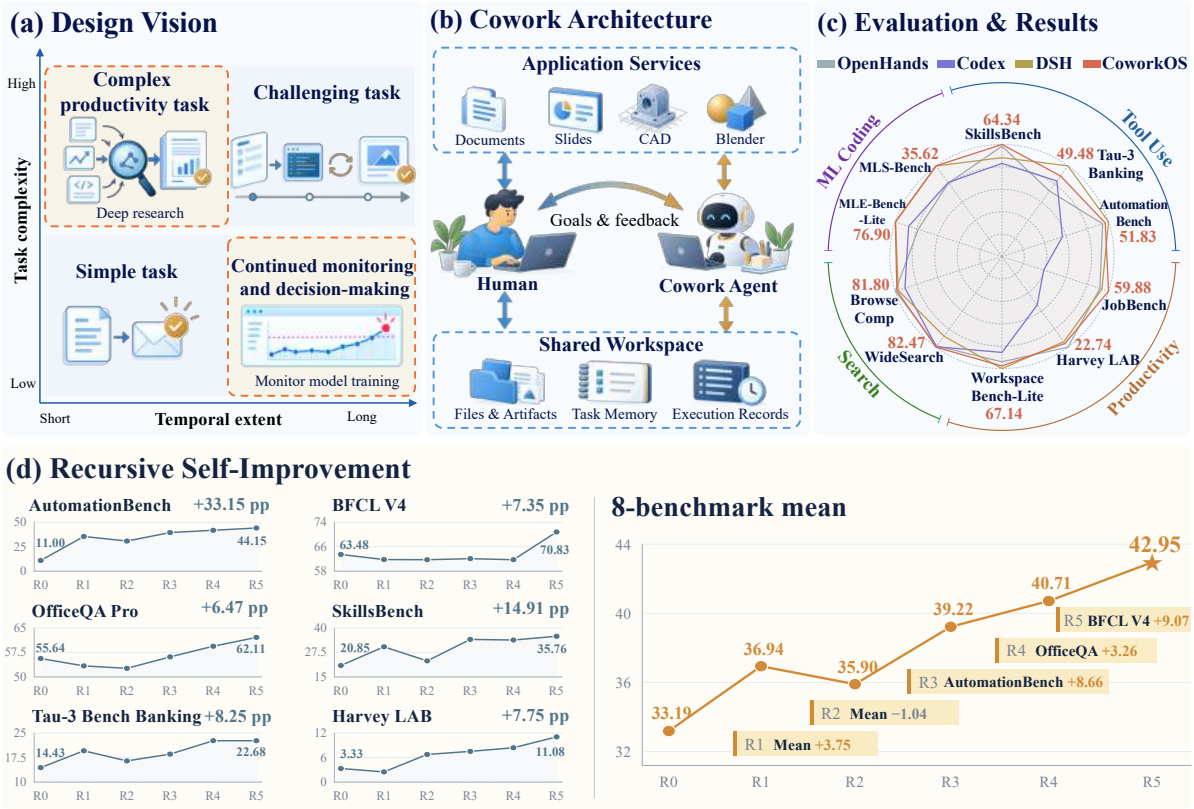


Figure 2: Overview of CoworkOS. **(a)** Two complementary requirements for cowork agents: complex productivity tasks and continued monitoring and decision-making. **(b)** Human–agent cowork through shared, evolving work. **(c)** Performance overview of CoworkOS alongside other frontier cowork harnesses across 10 benchmarks in four domains. **(d)** Recursive self-improvement (RSI) via automated post-training over five improvement rounds. Details are provided in Section 4.3 and Appendix A.3.

decomposes work as needed, and evaluates the final artifacts and results returned. The Coordinator dispatches subtasks to the corresponding agents and monitors their lifecycles. Subagents carry out their assignments within the context and resources provided to them, using tools and making local decisions. Thanks to this hierarchy, CoworkOS can support complex productivity tasks through collective intelligence, while the Coordinator enables Subagents to sustain continued monitoring and decision-making on long-horizon tasks.

To realize this collaboration pattern at scale, we build a **human-in-the-loop scalable application ecosystem**. It provides foundational tool capabilities including CLI execution, search, file operations, and MCP interfaces, together with extensible access to professional applications, including document editors, PPTist, CAD, and Blender. These tools support work across code, office artifacts, three-dimensional content, and games. A distinctive feature is the shared application object: people and agents can inspect and revise the same evolving document, model, or scene. Domain operations, rendered views, and other supported feedback inform subsequent decisions, so human participation can enter both through conversation and through direct modification of the work and its artifacts, embedding human-in-the-loop participation within autonomous execution.

To sustain persistent memory and communication across agents, we introduce **hierarchical workspaces and persistent semantic memory**, which provide continuity for the task. Shared workspaces let agents inspect and reuse one another's working files and candidate artifacts; semantic memory records key decisions, constraints, and unfinished work; and long-running operations run as Jobs that retain their status and logs, so work can be picked up accurately after waiting or interruption.

As shown in Figure 2(c), we evaluate CoworkOS on 12 general agent benchmarks and across 6 categories of real-world application scenarios. CoworkOS shows competitive performance relative to strong mainstream cowork frameworks such as OpenHands, Codex, and DeepSeek Harness.

To further explore the potential of CoworkOS, we task it with **RSI-native automated model post-training**, a workload that naturally combines complex productivity work with continued monitoring and decision-making. Specifically, CoworkOS automatically analyzes model weaknesses, optimizes the data pipeline, synthesizes and validates post-training data, feeds it into training, and iterates this closed loop. Doing so requires the model not only to decide wisely how the data pipeline should be optimized, but also to monitor long-running training runs and preserve their execution logs. Over five improvement rounds, CoworkOS raises the average score of Qwen3.6-35B-A3B from 33.19 to 42.95 across eight challenging productivity benchmarks, a gain of 9.76 points, making strategic optimization decisions throughout the iterations and demonstrating its strong potential as an RSI-native coworker. Qualitative comparisons and post-training run records are detailed in Section 4.2 and Appendix A.

This report makes four contributions:

- **A cowork harness architecture combining complex productivity tasks with continued monitoring and decision-making.** A division of responsibility lets the model autonomously choose the task-execution strategy while the runtime harness maintains efficient, long-horizon cowork mechanisms; the three-layer framework of Main Agent, Coordinator Runtime, and Subagents supports complex productivity tasks through collective intelligence and sustains continued monitoring and decision-making on long-horizon tasks.
- **A human-in-the-loop scalable application ecosystem.** Foundational tools and extensible professional applications including documents, presentations, CAD, and Blender support code, office artifacts, three-dimensional content, and games; shared, evolving application objects let humans participate through conversation or by directly modifying the work itself, embedding human-in-the-loop participation within autonomous execution.
- **Persistent working state and execution continuity.** Hierarchical workspaces and persistent semantic memory keep working files, artifacts, decisions, and unfinished work shareable and reusable across agents, while Jobs retain the status and logs of long-running operations so work can resume accurately after waiting or interruption.
- **Cross-domain generalization with RSI-native capability.** Across 12 benchmarks in four domains and six categories of real-world cowork scenarios, CoworkOS generalizes across tool use, productivity, search, and ML coding with performance comparable to strong mainstream cowork frameworks; in a five-round RSI-native post-training study, it further raises the average score of Qwen3.6-35B-A3B from 33.19 to 42.95 over eight benchmarks.

2 Preliminaries: Task Formulation and Agency

We first define the tasks an agent system is expected to solve, then describe our view of agency and two dimensions that characterize the challenges these tasks present.

2.1 General Agent Task Formulation

An agent task is specified by

$$\mathcal{T} = (g, \mathcal{E}, \mathcal{A}, \mathcal{V}, B), \tag{1}$$

where g is the user's objective, $\mathcal{E}$ the environment, $\mathcal{A}$ the action space, $\mathcal{V}$ the acceptance criterion, and B the resource budget. The environment can include information, editable artifacts, applications,

people, and running processes. Actions include tool and application operations, delegation, requests for human input, and delivery.

At decision boundary k, the task and environment state s_k evolves through agent actions and contributions from humans and ongoing processes:

$$\begin{aligned} s_{k+1} &= F_{\mathcal{E}}(s_k, a_k, u_k, \epsilon_k), \\ o_{k+1} &= O_{\mathcal{E}}(s_{k+1}, a_k), \end{aligned} \tag{2}$$

Here $a_k \in \mathcal{A}$ is the agent action, u_k represents human instructions or direct edits, and ϵ_k captures changes from external sources and ongoing processes. The agent receives observation o_{k+1} through tools, applications, or human feedback and chooses subsequent actions based on the objective and available interaction history. Humans can participate throughout the task, both by guiding decisions and by editing the same artifacts on which the agent works.

For an interaction trace τ, the objective is to maximize the probability of acceptance, $\mathcal{V}(\tau) = 1$, within budget B. Acceptance can assess delivered artifacts and required responses over an agreed period, using tests, external evaluation, or human judgement.

2.2 Our View of Agency

We view *agency* as a system's ability to choose and carry out actions that advance a user's objective, adapting its approach to observations and human guidance (Sumers et al., 2024). The challenge of exercising this ability depends on both the structure of the work and how long the system must remain responsible for it. We characterize these demands along two dimensions (Figure 2(a)).

Task Complexity. This dimension captures the number and interdependence of decisions, the branches that must be explored, and the tools and evidence that must be combined to produce an acceptable result (Liu et al., 2024). Software development, research, and design can demand substantial planning, investigation, and verification even when little time is spent waiting for external events. These demands remain even if inference and tool execution become arbitrarily fast. Complex delivery primarily challenges agency along this dimension.

Temporal Extent. This dimension captures how long the objective remains active, including periods awaiting computations, human input, or changes in the environment. A monitoring task may require few decisions while remaining relevant for days or weeks (Sumers et al., 2024). Sustained supervision therefore calls for the ability to retain the objective, recognize when new information warrants action, and respond as conditions change.

Tasks can be demanding along either or both dimensions. Persistent monitoring can combine a long elapsed horizon with relatively simple local decisions, while sustained project work can require both complex problem solving and continued responsibility. Recursive self-improvement (RSI) illustrates their intersection (Zelikman et al., 2023): an agent analyzes weaknesses and proposes changes, waits for training and evaluation, then uses the results to guide further work. CoworkOS targets tasks across these dimensions, with a common framework for complex delivery, sustained supervision, and their combination.

3 The CoworkOS Agent Harness

In this section, we introduce CoworkOS, a cowork harness for long-horizon and human-in-the-loop task execution. Figure 3 shows three parts of the harness:

- **Hierarchical Agent Runtime Architecture:** a three-layer runtime with communication and coordination during execution.

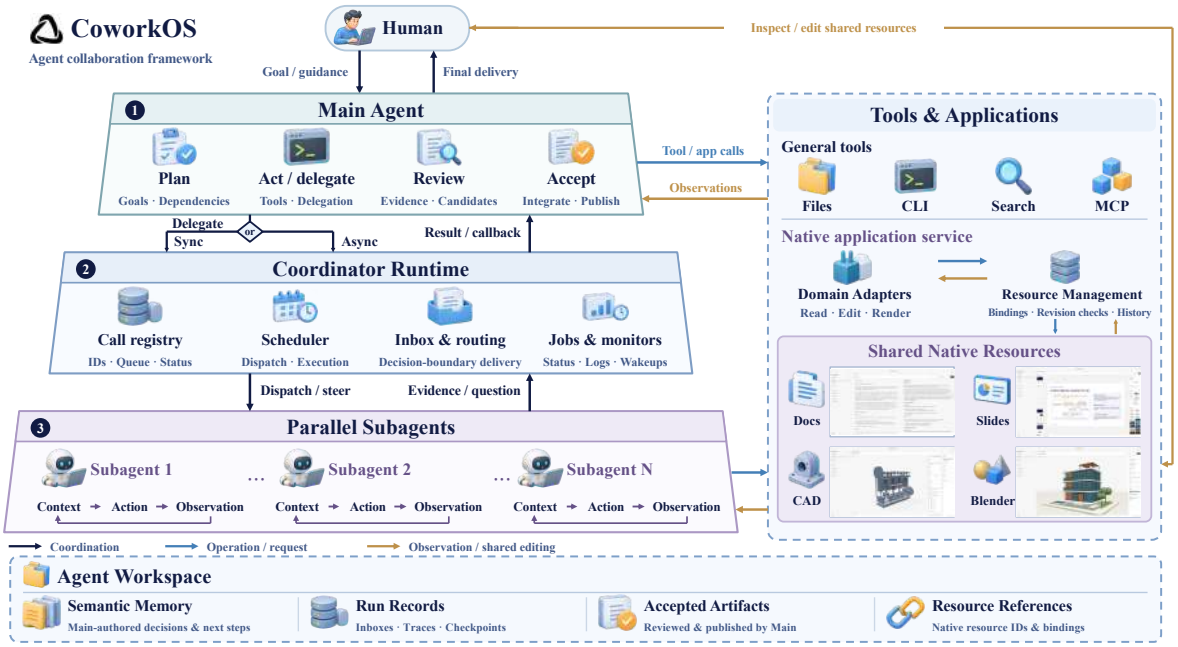


Figure 3: The CoworkOS harness and its shared action space. Main directs and reviews work, while the Coordinator Runtime manages synchronous or asynchronous Subagent execution and feedback. Main and Subagents share tools, application services, and an agent workspace. Asynchronous delegation lets Main continue other work. Application services support native resource editing, revision management, and conflict detection.

- **Scalable Application Ecosystems for Human–Agent Collaboration:** an application extension model and a shared resource protocol for human and agent contributions.
- **State Management for Persistent Agent Workspaces:** monitoring, checkpointing, workspace management, and context and memory mechanisms that preserve progress across executions.

3.1 Agent Runtime Architecture

Long-horizon coworking creates a tension between global coherence and local execution. The system must preserve one accountable interpretation of the user's objective, while allowing context-heavy investigations to proceed in parallel and recover from interruption. We resolve this tension by separating authority from execution: Main owns task-level decisions, Subagents own bounded assignments, and the Coordinator makes the exchanges between them durable and observable. This organization is implemented through a hierarchical runtime (3.1.1) and explicit communication paths (3.1.2).

3.1.1 Hierarchical Agent Organization

To preserve a coherent global context while enabling efficient parallel and independent task execution, we adopt a hierarchical agent organization with a centralized control structure centered on Main. Main serves as the user-facing interface and the overall planner, manager, and executor, and autonomously invokes Subagents when parallel processing or independent task handling is needed. This design is motivated by three considerations:

- the full task context remains in Main, allowing it to better integrate user instructions with the interaction history
- context-heavy tasks that can be handled independently can be delegated to Subagents with isolated contexts, improving efficiency without interfering with Main's context

- concentrating most responsibilities in Main facilitates learning effective task control and coordination.

Main Agent. Main is the system's point of accountability to the user. It interprets instructions, acts directly when the task requires global context, and revises ongoing work as the user changes objectives or constraints. To support this role, the harness gives Main a comprehensive toolset and access to skill documents, with relevant capabilities and instructions loaded as needed. This allows Main to investigate, execute, and verify work within its own decision loop. At decision step t, it selects an action according to

$$a_t^{\mathrm{M}} \sim \pi_{\mathrm{M}}\left(\cdot \mid g, h_t^{\mathrm{M}}, \mathcal{F}_t\right), \tag{3}$$

where g is the user's objective, h_t^{M} is the interaction history retained in Main's context, and $\mathcal{F}_t$ contains newly available feedback, including execution results and user guidance.

Beyond direct execution, Main can request human input and delegate scoped work through dedicated tools. A delegation is therefore an evidence contract: Main specifies the objective, constraints, capabilities, and form of evidence expected at the boundary of the assignment. This is useful for long tool interactions whose intermediate trajectory need not enter Main's context and for independent activities that can run in parallel. Main remains responsible for interpreting the returned contribution and deciding how it affects the task.

To keep this accountability loop responsive during lengthy operations, the runtime supports synchronous and asynchronous execution. A synchronous call keeps the result on Main's calling path. An asynchronous delegation or background command returns an acknowledgement and identifier, while its outcome is durably queued in Main's Inbox and incorporated at a later decision boundary alongside user guidance. Main can continue independent work while results are pending, or wait explicitly when a result is a prerequisite for acceptance.

Subagents. Each Subagent maintains a separate working context for a scoped assignment. It can conduct extended local investigation and tool use without adding every intermediate step to Main's task-level context. For assignment d_i, its initial context is constructed as

$$c_{i,0} = \Phi_{\mathrm{S}}\left(d_i,\ b(h_{t_i}^{\mathrm{M}}),\ \Gamma_i,\ \mathcal{S}_i\right), \tag{4}$$

where t_i denotes the dispatch step, $b(\cdot)$ extracts bounded recent background from Main, Γ_i specifies the granted tools and resources, and $\mathcal{S}_i$ is a compact catalog of available skills. The Subagent retrieves full skill instructions and additional working materials as needed.

Within this scope, the Subagent chooses and revises a local sequence of actions as observations arrive, forming a trajectory τ_i. It may explore alternatives, construct candidate outputs, and verify them with the granted capabilities. It returns a contribution $r_i = (y_i, \mathcal{P}_i, \sigma_i)$ consisting of a report, references to outputs and supporting records, and execution status. The report states what was done and what remains uncertain, which makes the result inspectable without importing the entire trajectory into Main.

Coordinator Runtime. The Coordinator is the control-plane boundary between these contexts. On delegation, it assembles the assignment, bounded background, and granted capabilities into a worker context. On return, it exposes the report, evidence references, and compact execution status to Main while keeping the detailed trajectory separately accessible. The Coordinator thus moves evidence across context boundaries without merging the contexts themselves.

These information exchanges can span multiple model decisions and interruptions, so their ownership and delivery state must persist beyond the current context. The Coordinator therefore maintains

assignments, execution state, and pending feedback in persistent program state. Let ξ_k denote this state after event k. Its evolution follows

$$\xi_{k+1} = F_{\mathrm{C}}\left(\xi_k, e_{k+1}\right), \tag{5}$$

where e_{k+1} represents a delegation request, execution update, message, or terminal result, and F_{C} is the durable runtime state-transition logic. Stable identities associate guidance and returned results with the corresponding assignment and Task. Queues, leases, and acknowledgements preserve ownership and delivery state across model decisions and recoverable interruptions, so Main can continue from operational evidence rather than from an assumed in-memory history.

3.1.2 Agent Communication and Coordination

Information flows through three directional paths in the hierarchy: Main-to-Subagent, Subagent-to-Subagent, and Subagent-to-Main. We represent the communication graph as

$$\mathcal{G}_t = \left(\mathcal{V}_t,\, \mathcal{E}_t^{\mathrm{hier}} \cup \mathcal{E}_t^{\mathrm{peer}}\right), \tag{6}$$

where $\mathcal{V}_t$ contains Main and the Subagents participating in the Task, $\mathcal{E}_t^{\mathrm{hier}}$ contains interactions involving Main, and $\mathcal{E}_t^{\mathrm{peer}}$ contains direct Subagent–Subagent exchanges.

Main-to-Subagent Transmission. Main starts a delegated execution with a concrete objective, relevant background, constraints, expected evidence, and explicit capability grants. The Coordinator assigns a stable identity, creates the scoped worker context, and binds it to the originating Task. This handoff transfers enough context for local decisions without copying Main's complete history. During execution, Main may enqueue a focused correction, new constraint, or follow-up requirement. The worker queues guidance and incorporates it at a safe model decision boundary. Cancellation uses a separate control path that can terminate the execution.

Subagent-to-Subagent Transmission. Some assignments cannot be fully isolated: a worker may depend on an interface, assumption, or candidate held by another worker. We therefore provide a minimal peer channel within a Task and Run. A Subagent sends concise text or a file/artifact reference to a non-terminal peer through the Coordinator. The Coordinator persists it in the recipient's Inbox and returns a message identifier with queued status; at a later safe decision boundary, the recipient sees the sender identity with the message. Queueing and delivery are recorded, while Inbox state tracks consumption. Messages to terminated targets are rejected or marked undelivered. The channel is asynchronous and non-blocking, and a response is optional and expressed as another peer message. It resolves local dependencies without turning peer exchange into a second control hierarchy: Main still owns task scope, further delegation, and final acceptance.

Subagent-to-Main Transmission. When a Subagent completes or fails, it returns a report, execution status, and references to outputs, diagnostics, and supporting records. With synchronous delegation, the result remains on Main's calling path. With asynchronous delegation, the runtime first returns an acknowledgement and call identifier, allowing Main to continue independent decisions; the terminal result later enters Main's Inbox and can trigger a subsequent Main turn. Main may wait for selected assignments or for all outstanding assignments when their results are prerequisites for acceptance.

An asynchronous Subagent may request a concrete decision from Main when an ambiguity changes its approach. The request carries a bounded wait and a declared fallback assumption, so the worker can make progress without holding the runtime indefinitely. Completed contributions, decision requests, user messages, and relevant background events enter Main through the same task-level integration path. Subagent contexts do not receive user inputs directly; Main assesses their effect on the global objective and decides whether to revise, continue, or cancel delegated work.

Message Delivery and Shared Evidence. Across the three paths, a message carries the smallest useful unit of coordination and points to evidence that can be inspected separately. The Task workspace exposes selected references and accepted artifacts without granting workers unrestricted write access (Figure 4). The Coordinator associates each message with its Task or assignment, delivers it at a model decision boundary, and preserves deferred messages for recovery. Agents therefore exchange focused information while retaining a path to the evidence needed for local execution or task-level integration.

Synchronous results stay on the calling path. Asynchronous work returns an acknowledgement and identifier before its terminal result enters the recipient's Inbox. The durable work queue records assignment and execution state; leases and acknowledgements allow pending feedback to be recovered without duplicating callbacks or contributions. These mechanisms let focused contexts progress independently while preserving one task-level interpretation in Main.

3.2 Scalable Application Ecosystems for Human–Agent Collaboration

Long-horizon coworking rarely stays inside one application. The same artifact may be discussed in conversation, edited by a user, inspected by one agent, and refined by another. The central integration problem is therefore continuity of resource state: each participant must act on a known version of the artifact, and a contribution must remain attributable when the artifact changes underneath it. We build an application ecosystem inside CoworkOS around this invariant.

To make this ecosystem extensible, we first define a common resource protocol for application integration. It gives the runtime a shared way to identify resources, apply and validate edits, and preserve revisions as work moves between users, agents, and successive executions. On top of this protocol, we support two integration paths: native applications managed by CoworkOS and existing software connected through MCP. The two paths differ in how capabilities enter the workspace while sharing the same Task and resource lifecycle. Figure 4 shows collaboration through a native application resource.

3.2.1 Resource Protocol

For a platform-managed resource, identity and revision are first-class because an edit without a base state cannot be evaluated after concurrent work. We represent it as $r = (\iota, \kappa, x, \upsilon)$, where ι is its identity, κ its kind, x its application-owned content, and υ its committed revision. The resource has a lifecycle independent of any one conversation or application window; Tasks and successive Runs hold references to the same evolving artifact.

The platform manages identity, references, revisions, and history, while each adapter preserves the semantics of its domain. Text edits and CAD parameter updates can therefore remain different operations while following one admission rule. For a new operation u that passes domain validation,

$$\mathrm{Commit}(r, u, \hat{\upsilon}, \alpha) = \begin{cases} (\iota, \kappa, T_\kappa(x, u, \alpha), \upsilon + 1), & \hat{\upsilon} = \upsilon, \\ \text{conflict}, & \hat{\upsilon} \neq \upsilon, \end{cases} \tag{7}$$

where $\hat{\upsilon}$ is the operation's base revision, α identifies the human or agent actor, and T_κ applies the operation according to the adapter's semantics. Both human and agent edits follow this rule. An accepted change advances the revision and records its actor and summary. A stale proposal returns a conflict rather than silently overwriting intervening work; the proposal remains available for reconsideration against the current state. Revision control is thus the mechanism that turns mixed human–agent editing into an auditable sequence of accepted changes.

Main can grant a Subagent permission to read a resource or prepare a private candidate. The candidate remains separate from the platform-managed artifact while the Subagent works, so exploration

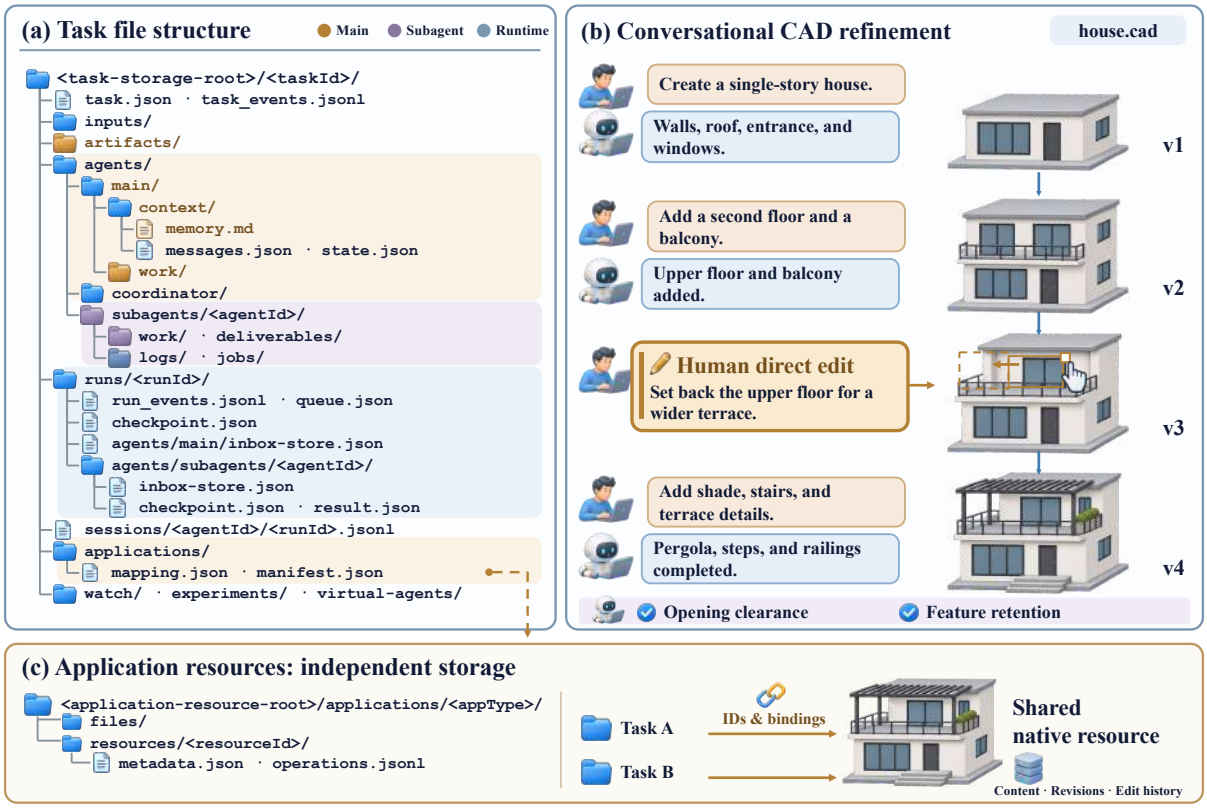


Figure 4: Task workspace structure and collaborative editing. **(a)** Selected file paths and their owners or record managers across agent, Run, and session scopes. **(b)** Iterative CAD refinement through user instructions, agent updates, and direct human edits to a shared model. **(c)** Task references and bindings link to independently stored application resources with persistent content, revisions, and edit history.

does not become an unreviewed shared edit. The Subagent returns the candidate with a report and supporting evidence; Main decides whether it should enter the artifact. The protocol then rechecks the candidate against Equation 7. If the baseline has changed, the candidate is preserved for reconsideration instead of being applied implicitly.

3.2.2 Native Applications

Native applications keep the resource under the platform's control, which lets direct human editing participate in the same coordination loop as agent actions. A user can revise a paragraph, move an object, or change a parameter to express a preference; the agent then observes that committed state through domain tools. Conversely, an accepted agent change updates the editor. Human edits are therefore task guidance expressed through the artifact itself, rather than a separate instruction channel.

The agent's application context includes the current resource and changes it has yet to acknowledge. For agent a, we write

$$c^{a}_{t,\mathrm{app}} = \Phi_{\kappa}\left(r_t, s_t, \Delta^{\neg a}_{v_a:v_t}\right), \tag{8}$$

where v_t is the current revision, v_a is the agent's last acknowledged revision, s_t is the saved selection, and $\Delta^{\neg a}_{v_a:v_t}$ contains intervening changes by other actors. The application supplies a bounded summary through Φ_{κ}; the agent retrieves detailed content through domain tools when needed. Change summaries exclude the agent's own edits, which are already represented in its interaction history. Human contributions can therefore enter the working context without loading the full application state at every decision.

Explicit application binding keeps the agent's scope focused as the workspace gains integrations. A binding associates the current resource with its tools, skills, and workflow instructions; when the binding changes, the runtime updates this scope and deactivates the previous application's tools. This prevents capabilities from unrelated applications from silently entering a decision context.

Domain operations address existing objects and parameters and return feedback on their effects. In Blender, stable object identities support targeted changes, and Python execution retains the existing scene by default. An agent can construct a scene, a user can adjust and save an object's transform or material, and the agent can read the updated scene before refining other objects (Figure 5). In CAD, editable source and parameter values preserve the design; geometry diagnostics and rendered views give both participants evidence for evaluating further changes.

Our native applications cover text documents, presentations, parametric CAD, and Blender projects. Each adapter supplies the editing and inspection operations for its domain. Import and export connect these editable resources to ordinary files and Task deliverables.

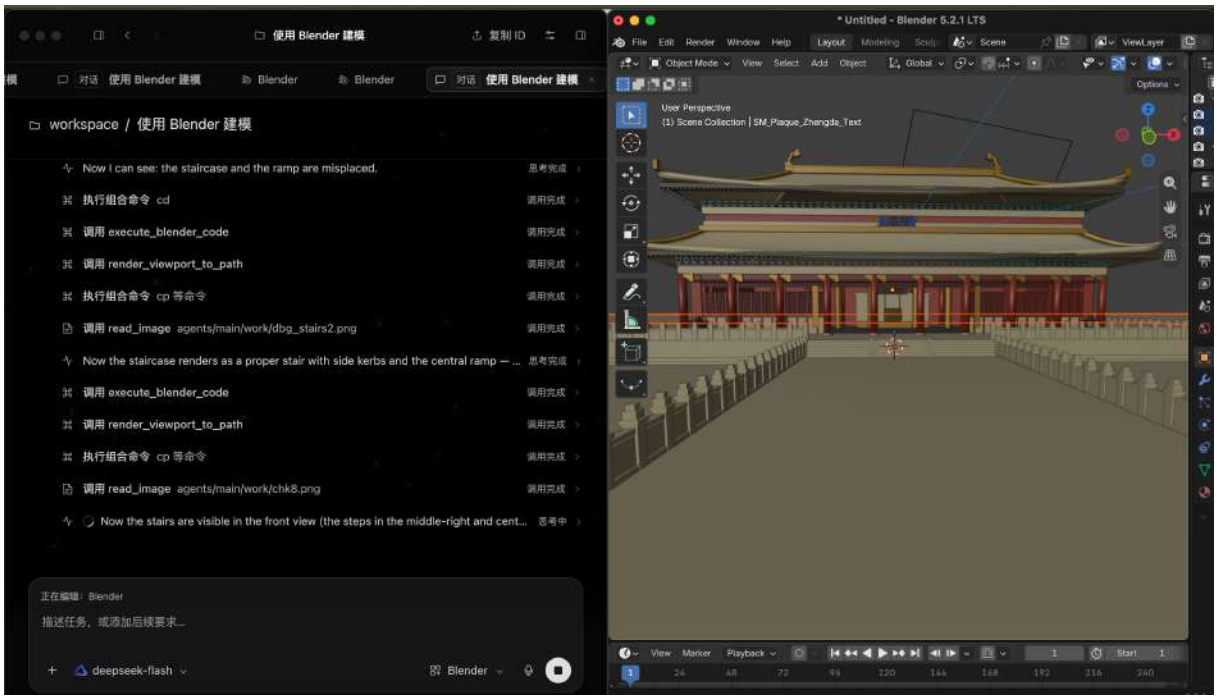


Figure 5: Blender integration in CoworkOS. The agent conversation (left) records code execution, updates to workspace files, and image inspection. Blender's native viewport (right) shows the resulting palace model in an editable 3D scene.

3.2.3 MCP-Integrated Software

MCP integration preserves an existing application's ownership and interface while making its capabilities available inside the Task. A plugin contributes the software's tools, domain skills, and workflow instructions, together with an adapter and interface integration that connect it to the runtime. The application can evolve independently while the Task still records which session and capabilities were used.

Selecting a software session binds its capabilities to the agent. The application owns the MCP service lifecycle, so the session can persist across individual agent executions, and users retain access to its interface throughout the work. Our integrations use external native windows for FreeCAD and Blender and embedded native views for CAD and OfficeHarness alongside the ongoing Task.

Document ownership depends on the integration. An MCP service that returns declared resource operations can participate in the shared protocol, including revision checks and history. An integration

that directly controls external software instead binds a session descriptor; the application retains its own document, saving, and concurrency semantics. This boundary keeps the platform from claiming guarantees that an external application does not expose.

Browser and desktop interaction are an orthogonal interaction mode for workflows that require graphical interfaces, rather than a third resource protocol. Agents use semantic or visual observations to operate Task-owned browser pages or a selected desktop window, then inspect subsequent observations or resulting artifacts. Users can pause desktop control and take over the window, moving between automated operations and direct interaction within the same Task.

3.3 State Management for Persistent Agent Workspaces

A long-horizon Task must retain progress across executions and supply a focused context for each decision (Shinn et al., 2023; Sumers et al., 2024). Building on the task formulation in Section 2.1, we preserve ongoing work and deferred feedback in runtime state, keep artifacts and evidence in the workspace, and reconstruct the agent's context from memory and current task state.

We represent retained Task state as $z_k = (W_k, P_k, R_k)$, where W_k contains working objects and artifact references, P_k records progress, decisions, and evidence, and R_k records ongoing executions, waiting conditions, and pending feedback. A Run records one concrete execution within the Task.

3.3.1 Persistent Execution

The runtime stores R_k independently of the model process, keeping ongoing work, waiting conditions, and pending feedback with the Task across turns and restarts. Computations and observation sources can run while Main is idle. The runtime resumes Main when a result, external event, or scheduled review calls for another decision.

Background Execution and Monitoring. Long computations run as background Jobs with persistent identities, process records, and logs. A training run or build can continue on the running host across conversation turns, context resets, and Main restarts. Main inspects process liveness, elapsed time, log activity, and recent output to decide whether to wait or intervene. If a process disappears without a recorded exit, its outcome remains uncertain; its logs remain available for investigation.

Deterministic source scripts monitor external state by comparing samples with a saved baseline. Unchanged observations require no model invocation. Candidate events go directly to Main or undergo semantic analysis first. Sampling proceeds independently of model analysis, and source cadence is separate from Main's scheduled goal reviews, allowing frequent observation with few idle model calls.

Before parking, Main records its progress, waiting conditions, and intended next action. A continuing goal must have an active observation source or an explicit review time; the runtime rejects a wait that provides neither. In-session goals retain the current Task, while independent background goals have their own Task and Main. Local services run the observation sources and persist source and goal state for continuation after a restart.

Checkpointing and Recovery. A source commits its updated cursor together with emitted events and retains those events until a durable Inbox accepts them. Stable identities suppress duplicate delivery during replay. The runtime acknowledges feedback after it enters Main's saved context (Section 3.1.2), so observations that Main has not yet processed remain available after a restart.

At completed turns, the runtime checkpoints the agent's message trajectory, compaction summary, and the position separating summarized and retained messages. An unfinished Run can resume from this state after a recoverable interruption; Main's session history and context snapshots also support task-level recovery. A supervisor reactivates an absent Main when eligible work is pending. It coordinates execution ownership and respects explicit pauses and waits for human input. External

work may have progressed since the checkpoint, so the resumed agent can inspect current process, artifact, or application state before deciding whether to repeat an operation.

The runtime checks for active Jobs and delegations, undelivered feedback, unanswered requests, and recorded wake conditions, together with a completed assistant response, when assessing Task completion. Main evaluates whether the result satisfies the objective. A finished response can therefore leave the Task waiting for work that still requires attention.

3.3.2 Workspace and Artifact Management

The workspace retains working objects W_k alongside the progress records and supporting evidence in P_k. Agent working materials, Run records, and published artifacts occupy separate scopes under the Task (Figure 4). Agent working areas persist across Runs, so later executions can reuse existing files. Each Run retains its own events, checkpoints, and results for inspection of earlier work and interrupted attempts.

Agents can inspect files, contributions, and execution records throughout the Task. By default, they write only to their own working area; explicit capabilities can extend that scope to a project being edited. An agent can read another agent's findings while developing its own drafts and intermediate outputs, retrieving detailed evidence through shared references as needed.

Subagents place reusable outputs in their delivery areas and return references with conclusions and supporting evidence. Main reviews these candidates, integrates contributions as needed, and publishes selected results to the Task artifact area. Later assignments can start from reviewed artifacts and inspect the original candidate files or their producing Run records when needed.

Native application resources have their own storage, revisions, and lifecycle. The Task keeps references to them, and contributions follow the review and update protocol in Section 3.2. Both files and application resources remain available when the agents and contexts working on them change.

3.3.3 Memory and Context Management

The agent uses persistent Task state and available history h_k within a bounded decision context:

$$c_k = \Phi(g, h_k, z_k), \qquad \text{tokens}(c_k) \le L, \tag{9}$$

where Φ selects and assembles information for the current decision, and L is the context capacity. To reconstruct Main's context as the history grows, we combine the objective and consequential decisions in durable memory with current runtime and goal state. Detailed evidence remains accessible in the workspace for decisions that require more than the reconstructed context contains.

Task Memory. Main maintains a memory document with the objective, completed work, next actions, and important information or constraints. The document preserves decisions needed in later phases. A compact runtime view supplies execution status, pending feedback, and waiting conditions, allowing Main to reconcile its remembered plan with work that is still active or has since returned. Optional user preferences and project experience enter through scoped projections with a context budget. Project skills are listed compactly and loaded in full as needed.

Context Reconstruction and Compaction. At phase boundaries, Main can refresh its context for the work ahead. Before requesting a reset, it is instructed to update its memory with the progress and decisions needed for continuation. Once the current tool batch settles, the runtime applies the reset and constructs the task context as

$$c^{\mathrm{M}}_{t,\mathrm{task}} = \Phi_{\mathrm{ctx}}\left(m_T, \rho_T(t), d_T(t)\right) \oplus h^{\mathrm{M}}_{t,\mathrm{recent}}, \tag{10}$$

where m_T is Main's semantic memory, $\rho_T(t)$ projects the runtime and structured goal state, $d_T(t)$ summarizes recorded experiments when available, and $\oplus$ concatenates retained recent history. An explicit reset starts from the reconstruction summary with $h^{\mathrm{M}}_{t,\mathrm{recent}} = \varnothing$; automatic compaction retains a recent segment at a safe turn boundary. When the summary is insufficient, Main can inspect workspace materials and full interaction logs. It can also read current application state to account for changes made during a pause.

Within an assignment, the runtime compacts a Subagent's execution trajectory as the estimated context approaches the model's capacity. It summarizes an older prefix while retaining recent interactions and effective system instructions. Before adopting the compacted state, it persists the summary and its message cursor together so both can be restored consistently with the execution checkpoint. If summarization fails, the previous history remains available.

Experimental History. A Task-level ledger records experiment objectives, delegated Runs, candidate variants, artifacts, and metrics with source references. It distinguishes measurements read from run-produced metric files from values stated only in narrative reports. Once multiple measured rounds are available, a compact digest of initial and recent results accompanies context reconstruction and callbacks for the experiment. Main can compare these results, inspect the associated artifacts, and select the next experiment after a context reset. Section 4.3 evaluates this recursive self-improvement loop.

4 Experiments

We evaluate CoworkOS through standardized benchmarks, real-world application cases, and iterative improvement experiments. Section 4.1 examines task performance across four benchmark domains; Section 4.2 considers the completeness, consistency, and usability of delivered artifacts; Section 4.3 studies aggregate progress and task-level changes across improvement rounds.

4.1 Benchmark Evaluation

4.1.1 Evaluation Setup

Benchmarks. To evaluate complex, long-horizon, end-to-end productivity work, we select twelve benchmarks covering four capabilities. **Tool Use** includes SkillsBench (Li et al., 2026a), Tau-3 Bench Banking (Shi et al., 2026), and AutomationBench (Shepard and Salimans, 2026), evaluating tasks carried out through skills, tools, and interactive operations. **Productivity** includes JobBench (Li et al., 2026b), Harvey Legal Agent Bench (Harvey AI, 2026), GDPVal (Patwardhan et al., 2025), OfficeQA Pro (Opsahl-Ong et al., 2026), and WorkspaceBench-Lite (Tang et al., 2026), covering business work, professional materials, and office tasks. **Search** includes WideSearch (Wong et al., 2025) and BrowseComp (Wei et al., 2025), assessing information retrieval and synthesis. **ML Coding** includes MLE-Bench-Lite (Chan et al., 2025) and MLS-Bench (Lyu et al., 2026), assessing machine-learning implementation and experiment execution. Together, these domains cover complementary stages of practical work, from acting on an environment and processing information to delivering a result.

Model Configuration. We evaluate GLM-5.3 with High thinking effort, a 256K-token context window, and a maximum output length of 64K tokens per response.

Harnesses. We compare CoworkOS with three mainstream open-source harnesses—OpenHands (Wang et al., 2025), Codex, and DeepSeek Harness (DSH)—under the above task and model configurations, reporting each benchmark's evaluation metric.

Benchmark-specific task scopes, scoring procedures, and execution limits are summarized in Appendix A.1. The scores describe the actual configurations used in these runs.

Domain	Benchmark	OpenHands	Codex	DeepSeek Harness	CoworkOS
Tool Use	SkillsBench	63.30	53.48	56.75	**64.34**
	τ^3-Bench Banking	40.20	46.39	**55.67**	49.48
	AutomationBench	50.50	30.33	**53.50**	51.83
	Average	51.33	43.40	**55.31**	55.22
Productivity	JobBench	55.02	23.42	56.15	**59.88**
	Harvey Legal Agent Bench	**23.75**	12.70	22.30	22.74
	GDPVal (Elo)	1122	925	1157	**1164**
	OfficeQA Pro	66.20	**66.30**	62.30	65.90
	WorkspaceBench-Lite	64.54	58.81	**68.76**	67.14
	Average	55.27	40.11	56.13	**57.54**
Search	WideSearch	81.43	82.05	72.48	**82.47**
	BrowseComp	**83.00**	75.50	80.80	81.80
	Average	**82.22**	78.78	76.64	82.14
ML Coding	MLE-Bench-Lite	60.40	67.28	76.70	**76.90**
	MLS-Bench	28.91	29.05	35.57	**35.62**
	Average	44.66	48.17	56.14	**56.26**

Table 1: Benchmark evaluation across twelve tasks in four domains. Each domain ends with an equally weighted Average over the listed benchmarks. GDPVal reports Elo over all 220 scheduled tasks; its tie-adjusted win rate (percent) enters the Productivity average (Appendix A.1). Bold and underlining mark the highest and second-highest scores per row, including ties.

4.1.2 Evaluation Results

Table 1 summarizes results across the four domains. CoworkOS ranks among the top two harnesses in every domain average, achieving the highest averages in Productivity and ML Coding while closely matching DSH in Tool Use and OpenHands in Search. It also achieves the best results on SkillsBench, JobBench, WideSearch, and both ML Coding benchmarks, demonstrating broad competitiveness across tool execution, knowledge work, information retrieval, and machine-learning experiments. Taken together, these results give CoworkOS the strongest overall performance among the four evaluated harnesses and provide empirical support for its ability to handle complex, end-to-end productivity tasks.

Tool Use. CoworkOS demonstrates strong performance across Tool Use, with a domain average of 55.22 that closely matches DSH's 55.31 and exceeds those of OpenHands and Codex. It ranks first on SkillsBench with a score of 64.34, highlighting its ability to complete tasks through skills. On Tau-3 Bench Banking and AutomationBench, it joins DSH in the top two, maintaining competitive performance in interactive tool use and automation. Together, these strengths span skill use, banking interactions, and automated execution, supporting the tool operations required by complex workflows across different task settings.

Productivity. For practical knowledge work, CoworkOS achieves the highest Productivity average of 57.54, ahead of DSH's 56.13, demonstrating broad strength across professional tasks. It leads all evaluated harnesses on JobBench with a score of 59.88 and ranks among the top two on Harvey, WorkspaceBench-Lite, and GDPVal, covering business, legal, and office scenarios. Its OfficeQA Pro performance is also close to that of OpenHands and Codex. This combination of results shows broad support for information processing and task delivery across diverse work settings, aligning with Cowork's goal of addressing practical productivity needs.

Search. Strong search performance is evident in both individual results and consistency across tasks. CoworkOS achieves the best result on WideSearch with a score of 82.47 and ranks among the top two on BrowseComp, outperforming Codex and DSH on both benchmarks while its domain average of 82.14 closely matches OpenHands' 82.22. These results demonstrate strong information acquisition that complements tool execution and professional work. For Cowork tasks requiring external information, search supplies evidence and context for subsequent analysis and artifact creation, connecting information retrieval, content integration, and final delivery within a broader workflow.

ML Coding. On both MLE-Bench-Lite and MLS-Bench, CoworkOS achieves the best results among the evaluated harnesses with scores of 76.90 and 35.62, respectively, demonstrating strong machine-learning programming and experiment execution capabilities. It joins DSH in the leading group on both benchmarks, with both harnesses outperforming OpenHands and Codex, showing a consistent advantage across the evaluated tasks. These results extend the capability coverage from general-purpose tools and knowledge work to specialized technical workflows. They also show that CoworkOS remains competitive in tasks requiring implementation and experimentation, supporting Cowork's ability to undertake end-to-end machine-learning work.

4.2 Qualitative Analyses on Real-World Tasks

We examine how CoworkOS turns task requirements into visually organized, interactive deliverables. Figure 6 covers 24 cases: four frontend websites, four games, four Office presentation tasks, eight interactive visualizations, two CAD assemblies, and two Blender scenes. The following category analyses are grounded in representative observations from inspecting the final deliverables. Content-page comparisons, inspection coverage, and additional interaction evidence are documented in Appendix A.2.

4.2.1 Frontend Websites

The courtyard-inspection and packing-feedback cases show how CoworkOS consolidates object information and gives a more specific explanation when an operation is blocked (Figure 6a; interaction details in Figure 9). In *Courtyard House*, all three implementations support Kitchen inspection and retain the viewing settings. Codex offers an eye-level courtyard view with settings in the bottom controls, while DSH labels the room area and notes that settings are preserved. CoworkOS brings area, dimensions, time, section height, and material scheme together in the room card, making them directly available during inspection. In *Ridge Pack*, Codex reports that the picnic blanket cannot fit and suggests removing a larger item or using a larger pack. The shown DSH view lists compartment coordinates for packed items, with the blanket still unpacked. CoworkOS's rejection message additionally explains why spare aggregate volume is insufficient: the space is fragmented, and no placement clears the neighboring objects. It then suggests reordering the items or removing a bulky neighbor.

4.2.2 Games

CoworkOS combines more detailed scene construction with readable gameplay feedback (Figures 6b and 10). In *Paper Glider*, Codex emphasizes luminous rings and a corridor, and DSH adds a desk, lamp, and rug; CoworkOS organizes bookcases, curtains, seating, ceiling beams, and shadows into a recognizable study. Its result dialog also explains the missed-ring allowance and provides a working retry action. In *Fruit Slicer*, all three show separated fruit and score feedback. Codex and DSH use simplified fruit against wooden backgrounds, while CoworkOS integrates detailed fruit interiors, blade trails, and juice effects with a bamboo-framed mountain dojo; normal pointer input visibly advances both the cut count and score.

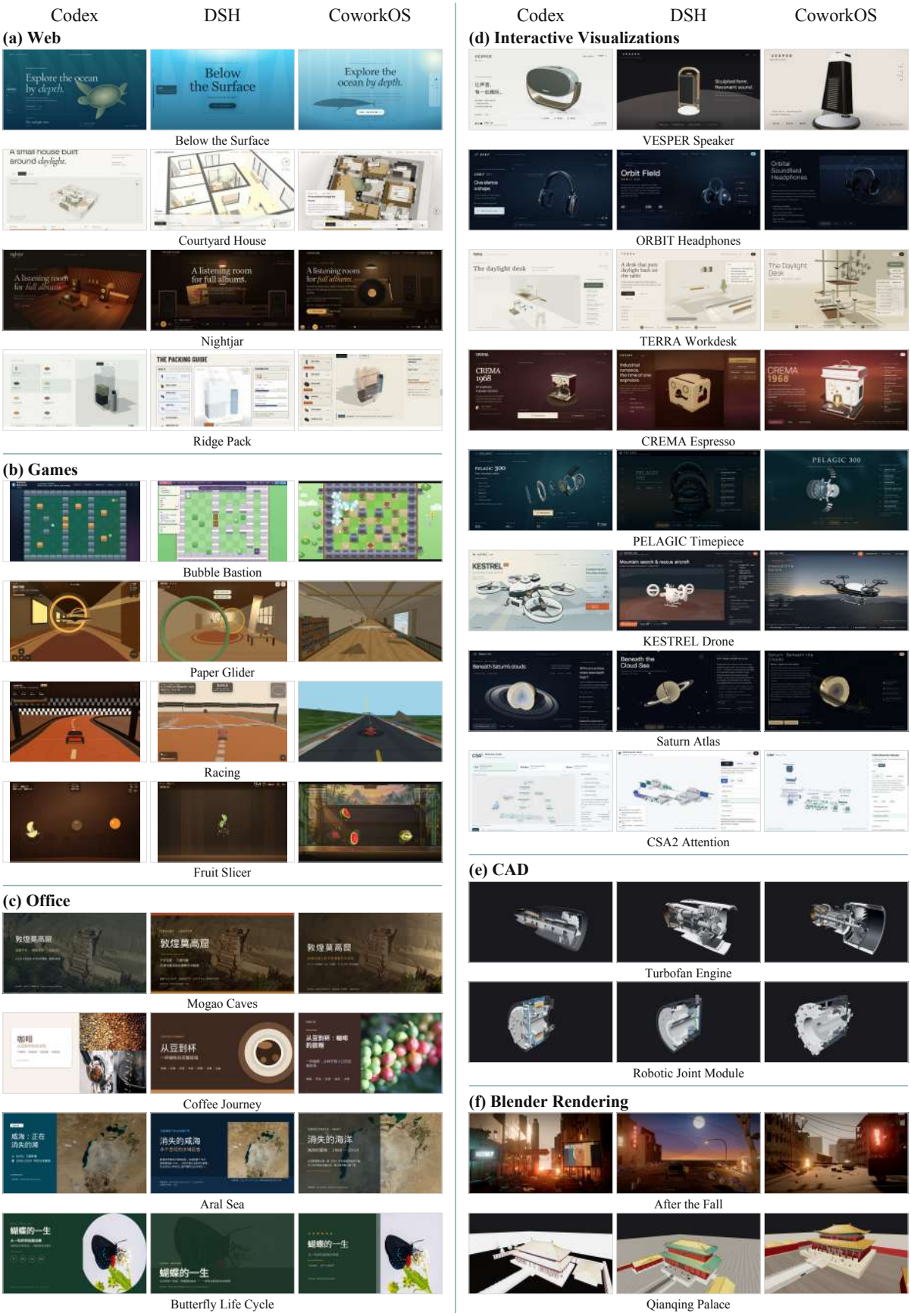


Figure 6: Qualitative comparisons across 24 cases: (a) frontend websites, (b) games, (c) Office presentations, (d) interactive visualizations, (e) CAD, and (f) Blender rendering. Each case shows Codex, DeepSeek Harness (DSH), and CoworkOS from left to right. Complete interactive artifacts are available in the online showcase.

4.2.3 Office

CoworkOS presents information more clearly through metric grouping and relation diagrams in the inspected Office pages. In *Mogao Caves*, Codex's descriptions extend below the metric-card borders, while DSH's quantity and duration suffixes wrap into the explanatory text. CoworkOS keeps each value, label, and explanation inside its card, making their association easier to follow. In *Aral Sea*, Codex's relation diagram places reduced inflow and evaporation sequentially in one chain, while DSH explains the causes in three text cards. CoworkOS instead draws them as separate branches converging on lake shrinkage, allowing readers to trace the two displayed paths directly. These observations support clearer grouping and connections in the shown pages. Content-page comparisons and page-level observations appear in Appendix A.2 (Figure 11).

4.2.4 Interactive Visualizations

In structural inspection and mechanism explanation, CoworkOS stands out in expanded-view framing and state legibility (Figure 6d). In *CREMA Espresso*, expansion moves the camera back and retracts the long introduction, separating the lid, boiler group, brew head, and base vertically while keeping component descriptions visible. Codex also provides a clear exploded assembly and eight descriptions; DSH requires manual zoom-out to fit the shell, which still partly occludes the boilers. The contrast is more direct in *CSA2 Attention*: in Reuse mode, CoworkOS presents selected positions in large front-facing labels, alongside yellow Main KV, coral Top-K, and an inactive indexing branch. Codex likewise shows cache preservation and inactive indexing, but its oblique position labels are smaller; DSH's low-angle layout crowds labels around Main Q, SWA KV, and Selection. These choices make geometry, provenance colors, and stage explanations easier to cross-reference within the displayed view. Operation details and case-specific limits appear in Appendix A.2 (Figure 12).

4.2.5 CAD

In the CAD cases, CoworkOS extends the core structure to present richer assembly detail (Figure 6e; detail views in Figure 13). In *Turbofan Engine*, Codex emphasizes the shaft, blade groups, and exhaust within a transparent casing. DSH already has detailed multistage blade rows in section, while its exterior mainly exposes annular bands and rods. By comparison, CoworkOS places bent tubes, annular components, and block-like accessories around the blade rows, making both the core and peripheral structure visible together. In *Robotic Joint Module*, its longitudinal ribs, populated flange fasteners, and side connection detail provide more explicit external assembly interfaces than DSH's smooth housing and flange holes; Codex likewise includes ribs, flange fasteners, and a layered interior.

4.2.6 Blender Rendering

Material distinctions and spatial depth are prominent strengths of CoworkOS's Blender artifacts in these cases (Figure 6f). In *Qianqing Palace*, Codex includes layered roofs, window divisions, and balustrades, but broad roof and wall surfaces appear pale. DSH provides colored tiles and painted eaves, with much of the main facade formed by continuous red door and window panels. CoworkOS distinguishes golden roofs, freestanding red columns, lattice panels, blue–green eave decoration, and pale stonework, with a visible separation between the colonnade and wall. In *After the Fall*, damaged multi-storey buildings enclose a street connecting foreground vehicles, a mid-street bus, and distant buildings; overhead cables and a hotel sign link the facades. Codex likewise creates a richly detailed urban corridor, using foreground signage, rubble, and street equipment to strengthen enclosure. DSH concentrates a rusty vehicle, low-rise ruins, and lighting around a more open shelter setting. These observations support distinguishable architectural components and connected environmental composition. Additional views and inspection limits appear in Appendix A.2 (Figure 14).

4.3 AI-Automated Post-Training

4.3.1 Motivation

Recursive self-improvement (RSI) offers an important direction for the continued development of AI capabilities: systems actively participate in optimizing their own capabilities and improvement mechanisms (Zelikman et al., 2023). Improved systems can then drive subsequent improvements, potentially forming a reinforcing feedback loop (Duan et al., 2026). AI-automated post-training is a representative practical direction within RSI, in which AI agents undertake the research and execution of model post-training to continually improve model capabilities.

This task brings complex delivery and sustained supervision together around a single model-improvement objective, exemplifying the sustained project work in Figure 2(a). Failure analysis, pipeline modification, and training decisions are interdependent and evolve as new evidence emerges; delayed training feedback and progress across rounds require continued maintenance of code, data lineage, and execution state. Both challenges closely match CoworkOS's core design: model-directed execution supports flexible research decisions, while continuity across contexts and time sustains long-running work. Automated post-training thus serves as an ultimate use case for demonstrating its support for complex, long-horizon tasks.

4.3.2 Setup

Models. CoworkOS organizes experiments and analysis. GLM-5.3 primarily serves as the controller and data teacher, while Qwen3.6-35B-A3B is the model trained and evaluated.

Initial state and iteration process. Starting from the base model, we perform seed supervised fine-tuning (SFT) on 1,345 tool-interaction trajectories from multiple sources to establish the R0 baseline, followed by five feedback-driven post-training rounds. Each round starts from evaluation of the current model and follows the cycle: **model evaluation → capability-gap diagnosis and data-pipeline revision → data generation, validation, and filtering → SFT → re-evaluation.**

Data quality and contamination control. All training data undergo BM25-based task-similarity filtering before training to control near-duplicate overlap between training and evaluation tasks.

Evaluation scope. We report results after each round on eight benchmarks: AutomationBench, BFCL v4, OfficeQA Pro (OfficeQA), Tau-3 Bench Banking, Harvey Legal Agent Bench, SkillsBench, GDPVal, and JobBench. GDPVal uses a normalized cumulative rubric score, separately from the Elo metric in Section 4.1. The aggregate is the arithmetic mean of the eight percentage scores, with each benchmark contributing up to 12.5 points. Further details appear in Appendix A.3.

4.3.3 Results

Across five post-training rounds, the model's average score on the eight benchmarks rises from 33.19 for the initial SFT model to 42.95, a gain of 9.76 points (Figure 8). The average increases consecutively from R3 through R5. The final model exceeds the initial baseline on seven of eight benchmarks and achieves the highest score across the six rounds on six, including ties (Figure 7), demonstrating CoworkOS's ability to sustain model improvement and broaden the gains from post-training.

Balancing multiple task capabilities is a central value of this result. Each round uses a single training dataset to update the same model, which must perform across all eight benchmarks; data selection and training strategies therefore need to accommodate different tool environments and behavioral requirements. Although later incremental supervision focuses on a subset of tasks, the final gains span function calling, complex tool execution, document question answering, and professional tasks, with seven benchmarks exceeding the baseline. This broad coverage demonstrates CoworkOS's ability



Figure 7: Per-benchmark performance across R0–R5 in RSI-native automated post-training. The 2×4 panels show Automation, BFCL v4, OfficeQA, Tau3 Banking, Harvey LAB, SkillsBench, GDPVal, and JobBench. Dashed lines mark the R0 reference for each task. Labels identify the initial and final scores and any intermediate maximum distinct from the final score. The horizontal axis denotes iteration round.

to balance targeted reinforcement with performance across tasks and provides positive evidence of cross-task generalization.

At the task level, Automation and SkillsBench show the largest gains of 33.15 and 14.91 points over R0, respectively, while Harvey LAB, Tau3 Banking, and OfficeQA show more sustained improvement. BFCL v4 gains 9.07 points from R4 to R5, consistent with the additional function-calling supervision in that round, whereas GDPVal and JobBench fluctuate overall. The best-performing round also differs by task: GDPVal peaks earlier, while Tau3 Banking matches its previous best in the final round and OfficeQA and BFCL v4 improve more substantially in later rounds. These differences reflect the changing trade-offs among tasks in joint post-training; the following analysis examines them through the evolution of the pipeline and training data.

4.3.4 Analysis

Across five rounds, CoworkOS continually uses evaluation feedback to identify capabilities to strengthen, revise the data recipe, and update the training strategy.

Identifying priorities from benchmark performance. The eight tasks do not respond uniformly to the same training round, and an increase in the overall average can coexist with stagnation on individual tasks. Per-benchmark results therefore provide a more specific basis for prioritizing data in the next round. Automation accounts for 78.3% of the training trajectories at R3 and continues to receive additional data in the following two rounds. Its performance improves consecutively from R3 onward, rising from 39.49 to 44.15 at R5 and ultimately yielding the largest gain among the tasks. BFCL, by contrast, remains below the initial baseline throughout R1–R4. At R5, the data emphasis shifts toward function calling, which accounts for 61.1% of the new trajectories, alongside a marked improvement on BFCL. The remaining 129 trajectories continue to cover complex operations, document question answering, and business interactions, retaining training support for other tasks while concentrating on function calling.

Improving demonstrations based on failure patterns. After identifying the tasks to strengthen, CoworkOS further adjusts demonstration content to address specific failed behaviors. R4 targets

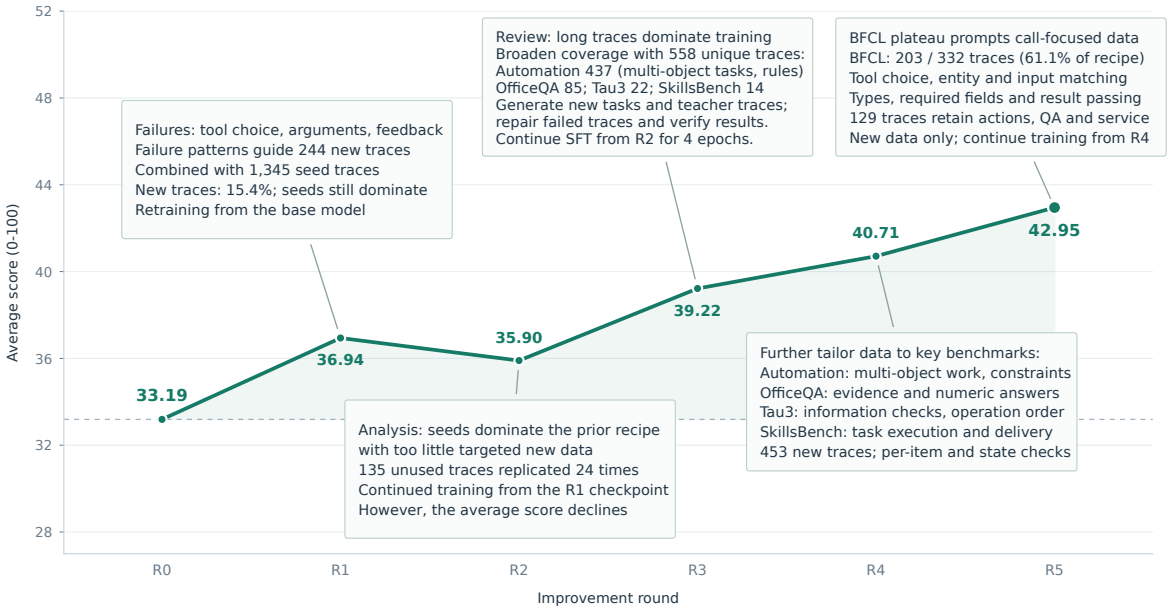


Figure 8: Average score across eight benchmarks and key changes in RSI-native automated post-training. The curve connects observed averages at R0–R5; the dashed line marks the initial seed-SFT baseline. Each annotation integrates the corresponding round's checkpoint start, epochs, optimization steps, data recipe, and strategy changes. Sampling multipliers are reported separately from epochs. R0 and R1 start independently from the base model; R2–R5 continue from the preceding checkpoint. These changes describe concurrent interventions rather than isolated causal effects; round index does not represent elapsed time. Detailed training records appear in Appendix A.3.

omissions in multi-item tasks, constraint violations, and missing verification after execution, requiring trajectories to demonstrate item-by-item handling, checks on operational constraints, and post-action state readback. Constraint adherence also becomes an acceptance criterion, leading to the exclusion of multiple Automation candidates at R5. For function calling, R5 further supplements demonstrations of entity and argument binding, parameter types, and required fields. These adjustments translate evaluation findings into concrete training requirements, with acceptance checks reducing the inclusion of erroneous demonstrations.

Planning training around training history and supervision distribution. To address R1's retraining from base and its high proportion of seed data, R2 selects the R1 checkpoint as its training starting point, stops replaying seed and previously trained data, and repeats 135 previously untrained trajectories 24 times. A review following the decline in average score from 36.94 to 35.90 finds that 42 trajectories contribute 84.2% of supervised tokens, giving a small number of long trajectories a large share of the supervision. R3 retains incremental fine-tuning, expands the independent set to 558 trajectories, and trains from the R2 checkpoint for four epochs, using broader data coverage to reduce reliance on extensive repetition of a small corpus. The average score recovers to 39.22. Training adjustments thus extend from whether data have already been used to the coverage, supervision distribution, and repetition intensity of new data.

These adjustments demonstrate CoworkOS's ability to accumulate evidence and revise experimental plans in complex, long-horizon research, providing empirical support for AI agents undertaking sustained model-development work.

5 Conclusion

We presented CoworkOS, a cowork harness for long-horizon, human–agent work. Its central design separates task strategy from execution management: Main selects and coordinates the work, while

the runtime manages delegated execution, communication, shared application resources, persistent workspaces, semantic memory, and long-running Jobs. This organization allows agents and users to continue working on the same evolving artifacts while preserving the state and evidence needed across interruptions and context changes.

Across 12 benchmarks and six categories of application cases, CoworkOS achieves competitive performance relative to OpenHands, Codex, and DeepSeek Harness. In an RSI-native post-training study, it raises the average score of Qwen3.6-35B-A3B from 33.19 to 42.95 over five rounds and eight benchmarks. Although the gains vary across tasks, the results support a unified harness for complex productivity work, continued monitoring, and iterative model improvement.

6 Author Contributions

CoworkOS is a collaborative project. We summarize the contributors' primary roles below.

Core Contributors

Jiajie Jin[1*], Guanting Dong[1*], Chengjun Pan[3*], Xin Deng[2], Zhuohui Sheng[2], Guoqiang Wang, Tao Gui[2†], Yutao Zhu[1†].

Contributions by Area

- **General Capability Optimization:** Yujia Chen[2], Shihong Yi[2], Binze Hu[2], Zhixiong Yang[2], Zhichao Sun[2], Rui Wang[2], Han Han[2], Mingyu Ma[2], Junhao Huang[4], Xiaotao Wang[2], Guoqiang Wang, Huicheng Jiang[2].
- **User Experience Optimization:** Yuyang Hu[1], Xinyu Yang[1], Wanghan Xu[4], Yujia Chen[2], Shihong Yi[2], Guoqiang Wang, Yajie Yang[2].
- **RSI Optimization:** Guanting Dong[1], Jiajie Jin[1], Zhuohui Sheng[2], Xin Deng[2], Zhixiong Yang[2], Han Han[2], Mingyu Ma[2].
- **Harness Development and Maintenance:** Jiajie Jin[1], Chengjun Pan[3], Yuyang Hu[1], Zhuohui Sheng[2], Guanting Dong[1].
- **Discussion and Support:** Zhecan Wang, Honglin Guo[2], Xiaoyu Xing[2], Xiaoqian Zhang[6], Yulun Zhu[2], Xiaoxi Li[1], Jiahang Lin[2], Shaofan Liu[2], Shihan Dou[2], Zhiheng Xi[2], Qi Zhang[2].

Project Mentorship

Tao Gui[2†], Yu-Gang Jiang[2], See-Kiong Ng[5], Yang Zhang[5], Tat-Seng Chua[5], Yutao Zhu[1†], Ji-Rong Wen[1], Zhicheng Dou[1†].

Affiliations

[1] Renmin University of China
[2] Fudan University
[3] Peking University
[4] Shanghai Jiao Tong University
[5] National University of Singapore
[6] East China Normal University.

* Equal contribution (co-first authors).
† Corresponding authors.

References

Anthropic. Claude Code: Overview. Official documentation, 2026. URL https://code.claude.com/docs/en/overview. Documentation accessed October 18, 2026.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Aleksander Madry, and Lilian Weng. MLE-bench: Evaluating machine learning agents on machine learning engineering. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

DeepSeek AI. DeepSeek Harness: Everything is a plugin. Official source repository, 2026. URL https://github.com/deepseek-ai/deepseek-harness. Repository accessed October 18, 2026.

Yi Duan, Ying Liu, Zirui Tang, Haodong Chen, Jun Zhou, Yumou Liu, Bangrui Xu, Yukai Wu, Sidi Chen, Yuhan Zhou, Haoyu Wang, Xiaoyou Yu, Shaokun Han, Xuzhou Zhu, Le Zhou, Bolin Lu, Wei Zhou, Jiachen Liu, Nuozhou Fang, Jiaxin Tian, Ruoyu Chen, Yuxuan Li, Kai Zuo, Kaiyan Zhang, Qianyu Yang, Zijie Wang, Jiantao Qiu, Conghui He, Guoliang Li, Bowen Zhou, Zhiyuan Liu, Zhoufutu Wen, Jihua Kang, Xuanhe Zhou, and Fan Wu. The last AI built by humans: Toward genuine recursive self-improvement. *CoRR*, abs/2609.11873, 2026.

Harvey AI. Harvey LAB: The legal agent benchmark. Official source repository, 2026. URL https://github.com/harveyai/harvey-labs. Accessed November 2, 2026.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023. URL https://arxiv.org/abs/2310.03714.

LangChain. LangGraph overview. Official documentation, 2026. URL https://docs.langchain.com/oss/python/langgraph/overview. Documentation accessed October 18, 2026.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianfu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han-Chung Lee. SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks. *CoRR*, abs/2602.12670, 2026a. doi: 10.48550/ARXIV.2602.12670. URL https://doi.org/10.48550/arXiv.2602.12670.

Yuetai Li, Yichen Feng, Zhangchen Xu, Zixian Ma, Kaiyuan Zheng, Fengqing Jiang, Xinghua Sun, Rulin Shao, Zichen Chen, Yue Huang, Xinyang Han, Brian Lee, Kayla Xu, Shenglai Zeng, Hang Hua, Xiangliang Zhang, Basel Alomair, Ranjay Krishna, Luke Zettlemoyer, Pang Wei Koh, Bhaskar Ramasubramanian, Luyao Niu, Xiang Yue, and Radha Poovendran. JobBench: Aligning Agent

Work With Human Will. *CoRR*, abs/2605.26329, 2026b. doi: 10.48550/ARXIV.2605.26329. URL https://doi.org/10.48550/arXiv.2605.26329.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating LLMs as agents. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

Bohan Lyu, Yucheng Yang, Siqiao Huang, Jiaru Zhang, Qixin Xu, Xinghan Li, Xinyang Han, Yicheng Zhang, Huaqing Zhang, Runhan Huang, Kaicheng Yang, Zitao Chen, Wentao Guo, Junlin Yang, Xinyue Ai, Wenhao Chai, Yadi Cao, Ziran Yang, Kun Wang, Dapeng Jiang, Huan ang Gao, Shange Tang, Chengshuai Shi, Simon S. Du, Max Simchowitz, Jiantao Jiao, Dawn Song, and Chi Jin. MLS-Bench: A Holistic and Rigorous Assessment of AI Systems on Building Better AI. *CoRR*, abs/2605.08678, 2026. doi: 10.48550/ARXIV.2605.08678. URL https://doi.org/10.48550/arXiv.2605.08678.

OpenAI. Codex CLI. Official documentation, 2026. URL https://learn.chatgpt.com/docs/codex/cli. Documentation accessed October 18, 2026.

Krista Opsahl-Ong, Arnav Singhvi, Jasmine Collins, Ivan Zhou, Cindy Wang, Ashutosh Baheti, Owen Oertell, Jacob Portes, Sam Havens, Erich Elsen, Michael Bendersky, Matei Zaharia, and Xing Chen. OfficeQA Pro: An Enterprise Benchmark for End-to-End Grounded Reasoning. *CoRR*, abs/2603.08655, 2026. doi: 10.48550/ARXIV.2603.08655. URL https://doi.org/10.48550/arXiv.2603.08655.

Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubeh, Phoebe Thacker, Laurance Fauconnet, Natalie S. Kim, Patrick Chao, Samuel Miserendino, Gildas Chabot, David Li, Michael Sharman, Alexandra Barr, Amelia Glaese, and Jerry Tworek. GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks. *CoRR*, abs/2510.04374, 2025. doi: 10.48550/ARXIV.2510.04374. URL https://doi.org/10.48550/arXiv.2510.04374.

Daniel Shepard and Robin Salimans. AutomationBench. *CoRR*, abs/2604.18934, 2026. doi: 10.48550/ARXIV.2604.18934. URL https://doi.org/10.48550/arXiv.2604.18934.

Quan Shi, Alexandra Zytek, Pedram Razavi, Karthik Narasimhan, and Victor Barrès. τ-Knowledge: Evaluating Conversational Agents over Unstructured Knowledge. *CoRR*, abs/2603.04370, 2026. doi: 10.48550/ARXIV.2603.04370. URL https://doi.org/10.48550/arXiv.2603.04370.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024.

Zirui Tang, Xuanhe Zhou, Yumou Liu, Linchun Li, Yukai Wu, Weizheng Wang, Hongzhang Huang, Wei Zhou, Jun Zhou, Jiachen Song, Shaoli Yu, Jinqi Wang, Zihang Zhou, Hongyi Zhou, Yuting Lv, Jinyang Li, Jiashuo Liu, Ruoyu Chen, Chunwei Liu, Guoliang Li, Jihua Kang, and Fan Wu. Workspace-Bench 1.0: Benchmarking AI Agents on Workspace Tasks with Large-Scale File Dependencies. *CoRR*, abs/2605.03596, 2026. doi: 10.48550/ARXIV.2605.03596. URL https://doi.org/10.48550/arXiv.2605.03596.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, and et al. OpenHands: An open platform for AI software developers as generalist agents. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents. *CoRR*, abs/2504.12516, 2025. doi: 10.48550/ARXIV.2504.12516. URL https://doi.org/10.48550/arXiv.2504.12516.

Ryan Wong, Jiawei Wang, Junjie Zhao, Li Chen, Yan Gao, Long Zhang, Xuan Zhou, Zuo Wang, Kai Xiang, Ge Zhang, Wenhao Huang, Yang Wang, and Ke Wang. WideSearch: Benchmarking Agentic Broad Info-Seeking. *CoRR*, abs/2508.07999, 2025. doi: 10.48550/ARXIV.2508.07999. URL https://doi.org/10.48550/arXiv.2508.07999.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation framework. *CoRR*, abs/2308.08155, 2023.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023b.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (STOP): Recursively self-improving code generation. *CoRR*, abs/2310.02304, 2023.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024. URL https://arxiv.org/abs/2410.10762.

A Evaluation Details and Supplementary Evidence

This appendix summarizes benchmark-specific evaluation settings, qualitative case evidence, and automated post-training run records.

A.1 Benchmark Evaluation Settings

SkillsBench (Li et al., 2026a). We use the 79 text-based tasks from SkillsBench, excluding visual and media tasks, with an independent workspace and task container for each instance. We retain the official task environments and verifiers and report the mean task score.

τ^3-Bench Banking (Shi et al., 2026). The evaluation contains 97 Banking Knowledge tasks, with BM25 retrieval, dense retrieval over 698 documents, and read-only shell access to the knowledge base. Each task allows up to 200 interaction steps, with a 1,800-second simulation limit and a 2,400-second outer task limit; we report task pass rate.

AutomationBench (Shepard and Salimans, 2026). We evaluate AutomationBench v1.0.6 with the null-type handling fix from PR #13. Tasks are scored with the benchmark evaluator, and we report the all-pass rate.

JobBench (Li et al., 2026b). We use the main split of 65 tasks, with a 7,200-second task timeout. Deliverables receive task-level percentage scores from an LLM judge, averaged over all 65 tasks.

Harvey Legal Agent Bench (Harvey AI, 2026). We use a fixed, proportionally sampled set of 120 legal tasks, retaining the official task documents, skills, and rubrics in isolated workspaces. Generated deliverables are graded criterion by criterion against the supplied rules and rubrics.

GDPVal (Patwardhan et al., 2025). For each harness, we include all 220 scheduled tasks and report Elo against a fixed baseline of 1000, computed as $E = 1000 + 400 \log_{10}[(W + T/2)/(L + T/2)]$, where W, L, and T are wins, losses, and ties. Tasks without a valid score count as losses, including candidate timeouts and call-gate exhaustion; thus $W + L + T = 220$ for every harness.

OfficeQA Pro (Opsahl-Ong et al., 2026). We use the 133-question OfficeQA Pro subset and its parsed Treasury Bulletin corpus, with a fresh workspace for each question. Answers are scored by the native EvalScope/OfficeQA rules.

WorkspaceBench-Lite (Tang et al., 2026). WorkspaceBench-Lite is built on the AgentCompass evaluation framework and uses the English Lite dataset. Each task is allocated 2 CPUs and 8 GiB of memory, with a two-hour candidate execution limit.

WideSearch (Wong et al., 2025). The task set comprises 200 questions, split equally between English and Chinese, with a shared Serper search and Jina page-reading backend. Answers are graded with the `format_normalized_v1` evaluator and an LLM judge.

BrowseComp (Wei et al., 2025). Tasks use web search, page reading, and Python, with a maximum of 500 interaction steps. We use discard-all context management and report answer accuracy.

MLE-Bench-Lite (Chan et al., 2025). We use all 22 low-split competitions, with a five-hour agent budget, 4 CPUs, 32 GiB of memory, and one H200 GPU per task. The official grader scores saved CSV submissions, which are converted to Human Rank using the retained competition leaderboards and equally averaged across the 22 tasks; missing or invalid submissions receive zero.

MLS-Bench (Lyu et al., 2026). We evaluate the 30 Lite tasks with a five-hour agent budget, task-specific images, and task-specific H200 GPU allocations. The standard per-task allocation is 4 CPUs, with 32 GiB of memory for CPU-only and single-GPU tasks and 64 GiB for multi-GPU tasks. Only the declared algorithm files may be modified; official verifier rewards are scaled from 0–1 to 0–100 and equally averaged across tasks.

Domain averages. Each domain average equally weights its listed benchmarks; GDPVal contributes the tie-adjusted win rate $100(W + T/2)/(W + L + T)$, calculated directly from the exact counts rather than rounded Elo. The remaining benchmarks use their reported 0–100 scores, with rounding applied only after aggregation.

A.2 Case Evidence and Inspection Protocol

Figure 6 covers 24 cases and 72 artifacts across six categories. Comparisons use unretouched images, shared aligned CAD inspection views, and authored Blender materials and lighting.

Frontend website evidence. Figure 9a compares Kitchen inspection in *Courtyard House*. The reviewer adjusts section height, time, and material before selecting the room; all three implementations provide local inspection and retain the settings on return. Codex pairs an eye-level courtyard view with section, time, and material controls at the bottom, without listing area or dimensions in the room description. DSH labels the 10.0 m^2 area and notes that daylight, section, and material are preserved. CoworkOS's room card combines 14.9 m^2, dimensions of 4.68×3.18 m, and the retained 17:00 time, 1.40 m section, and material scheme B. The comparison concerns how object information and current settings are presented; retaining settings is shared by the three implementations.

Figure 9b shows feedback in *Ridge Pack*. After attempts to insert the same eight named items in sequence, Codex reports that the picnic blanket cannot fit and suggests removing a larger item or using the 18 L pack. DSH's recorded view lists compartment coordinates for packed items, with the blanket remaining unpacked; its transient rejection message was not captured, so the record does not establish whether it explains the cause. CoworkOS's displayed message distinguishes spare volume from usable contiguous space: 5.98 L remains, but fragmentation prevents any placement from clearing neighboring items. It also suggests the Optimize action or removal of a bulky neighbor. This supports a more specific rejection explanation than Codex's visible message. The implementations use different authored item dimensions, so packed-item counts and capacity values are not used to rank packing algorithms.

Game scenes and interaction evidence. Figure 10 expands the *Paper Glider* and *Fruit Slicer* comparisons. The following checks cover inputs and feedback across all three harnesses in four games.

Bubble Bastion. Codex shows a cross-shaped splash, crate particles, and an increased crate-clearance counter; restarting clears this progress. DSH awards points after a bubble burst near a bush, presents a failure screen, and resets the score and enemy objective on retry. CoworkOS previews the cross-shaped danger area before the burst, repeats the objective in its pause panel, and restores the initial objective and hearts after a splash-related failure and retry.

Paper Glider. All three respond to steering, display ring progress, and provide failure and retry feedback. Codex reports missed target rings, while DSH couples ring passage with point notifications. CoworkOS's result dialog explains the missed-ring allowance; retry resets the progress. Figure 10a shows the scene differences: Codex foregrounds bright rings, windows, and a corridor outline; DSH adds a desk, lamp, rug, and plant. CoworkOS integrates book-filled shelves, curtains, seating, wall panels, ceiling beams, and floor shadows into a furnished study surrounding the flight route.

Racing. Codex and DSH register steering, checkpoint progress, and nitro consumption; restart resets checkpoint and timing displays. DSH also shows the vehicle and checkpoints on a minimap and

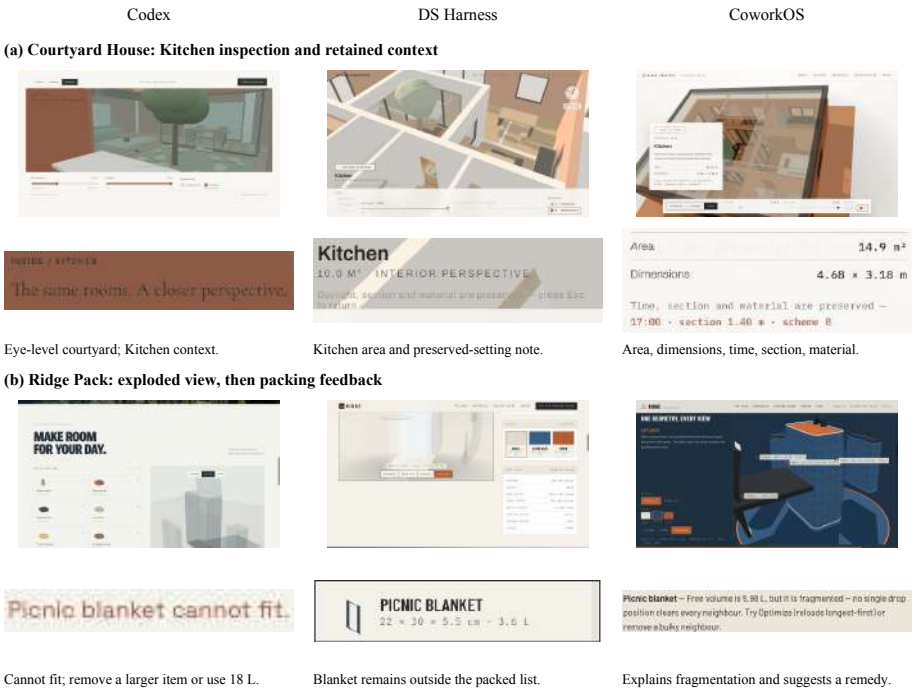


Figure 9: Frontend website interaction evidence. Columns show Codex, DSH, and CoworkOS. (a) Kitchen inspection with room information. (b) Exploded pack views, followed by feedback from the recorded item-insertion sequence. Each panel pairs a scene view with a native interface crop; English notes below the crops summarize the observations.

issues a wrong-way warning. In CoworkOS, throttle and steering change vehicle speed, heading, and distance to the next checkpoint. Pressing T after driving onto the shoulder returns the car to checkpoint 0 with an explicit message while retaining elapsed time; pressing R restarts the run and resets the timer.

Fruit Slicer. Pointer drags increase cut counts and scores in all three, with visible fruit separation and juice effects. Codex shows combo and double-slice feedback; DSH increments a missed-fruit counter before presenting failure and retry. CoworkOS combines detailed fruit interiors, blade trails, and particles with bamboo, mountain scenery, and a wooden dojo frame (Figure 10b). Its HUD reaches the cut target and invites continued scoring; the observed run subsequently ends after excessive misses. Retry resets progress while retaining the best score. Displayed scores and times reflect each game's own rules and are not used to rank harness success rates.

Office content-page evidence. Figure 11 preserves two sets of native content-page details used in the main comparison. In *Mogao Caves* (a), the descriptions in Codex's three metric cards extend below their bottom borders. In DSH's lower two cards, the suffixes meaning "more than" and "years" wrap below the large numbers and overlap the descriptions. CoworkOS keeps each value, label, and explanation inside its card. The pages use different figures and wording; the comparison concerns text containment and metric grouping, not numerical correctness. In *Aral Sea* (b), Codex's upper arrow chain sequentially connects upstream diversion, reduced inflow, evaporation, falling water levels, and ecological degradation. Its full page also separates human activities from natural conditions in two text groups below the chain. DSH organizes the process into three explanatory cards. CoworkOS separates reduced inflow and evaporation into two diagram branches entering a shared outcome. The distinction is whether the two displayed paths are separated directly in the relation diagram, not whether the baselines mention the relevant causes.

The Codex / DSH / CoworkOS page selections are 2 / 3 / 2 for Mogao metrics and 5 / 3 / 5 for Aral causes. Rectangular crops preserve complete metric groups, the arrow chain, the explanatory-card

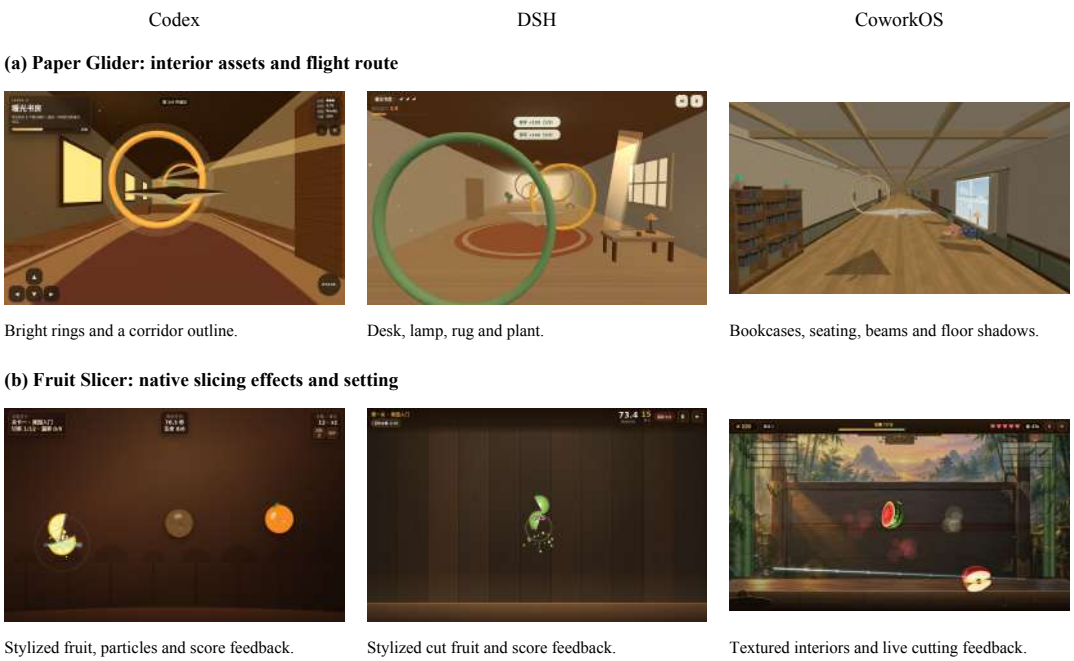


Figure 10: Game scenes and interaction details. Columns show Codex, DSH, and CoworkOS. (a) Interiors and flight routes in *Paper Glider*. (b) Slicing effects and environments in *Fruit Slicer*, including a frame from pointer-driven play. Native views are scaled proportionally without retouching.

group, or the branched diagram. The roasting pages in *Coffee Journey* provide a further comparison: CoworkOS page 6 links numbered photo annotations to descriptions of the roaster and cooling tray; Codex page 8 separates heating, moisture loss, cooling, and resting into four stage cards; DSH page 6 pairs stage descriptions with a time curve. The first emphasizes component-to-text correspondence, while the latter two organize the process. This does not establish broader topic coverage for CoworkOS. These content-page observations do not extend to *Butterfly Life Cycle*, for which only covers were inspected.

Interactive visualization evidence. In *CREMA Espresso* (Figure 12a), native expansion changes component positions and presents descriptions in all three implementations. CoworkOS moves the camera back, reduces the title, and retracts the long introduction, keeping the separated lid, twin-boiler group, brew head, drip tray, and base within one view alongside six description groups. Codex separates the lid and side panels in different directions and keeps eight descriptions beside the assembly. DSH's initial expanded view clips the upper shell; native zoom-out brings it into view, but the opaque shell still occludes parts of the boilers. The figure uses this zoomed-out view. This comparison concerns framing and information organization in the expanded state; it does not imply that the baselines lack expansion or component descriptions.

For *CSA2 Attention* (b), the reviewer switches from Full to Reindex and then Reuse at the Selection stage. Selected positions change from $[0, 3, 8]$ to $[1, 4, 7]$ and persist in Reuse. All three implementations display state changes, provenance colors, and stage explanations. CoworkOS uses front-facing labels to connect positions to yellow Main KV and coral Top-K, with text stating that corresponding entries are copied while the original cache remains. Codex likewise explains copy-forward behavior and cache preservation, and grays out inactive indexing; its in-scene position labels use a smaller oblique arrangement. DSH also explains index reuse, but its low-angle view crowds labels around Main Q, SWA KV, and Selection. These are legibility differences in the selected views, not validations of algorithmic equivalence or user comprehension. The advantage is also case-dependent: repeated



Figure 11: Office content-page details. Columns show Codex, DeepSeek Harness (DSH), and CoworkOS. (a) Mogao Caves metric groups, from pages 2 / 3 / 2. (b) Aral Sea cause explanations, from pages 5 / 3 / 5. Rectangular crops retain each complete metric group or relation representation; native text and graphics are unchanged.

reset, expansion, waiting, and rotation did not clearly expose separated internals in CoworkOS's *VESPER Speaker*, and Codex's *Saturn Atlas* cutaway distinguished interior layers more clearly.

CAD structure and native-geometry checks. Figure 13 supplements the main half-sections with assembled details. In *Turbofan Engine* (a), Codex shows axial blade groups and an annular core within a transparent casing, with relatively little peripheral equipment. DSH's opaque casing conceals the core at this angle, with annular bands and rods visible outside; the shared half-section in Figure 6e confirms its detailed multistage blade rows. CoworkOS distributes bent tubes, annular components, and block-like accessories around the core, supporting the main-text observation of richer peripheral assembly detail. In *Robotic Joint Module* (b), both Codex and CoworkOS have housing ribs and populated flange fasteners; the full assembled CoworkOS view also shows a protruding side connection detail. DSH uses a smoother housing with flange holes and a central opening, while its section still exposes a layered interior. These differences concern the visual representation of peripheral structure and interfaces; casing occlusion is not used to infer missing internals.

All six STEP files were independently imported and transferred with OpenCascade / OCP 8.0.1.1.0 without source modification. Visible axial cavities in section also do not establish a continuous through-bore in the complete assembly. These observations support native readability, inspectability, and comparisons of visible structural detail; they do not establish parametric editability, interference-free fit, tolerances, manufacturability, or operating performance. Orbit, section, and wireframe controls belong to the shared inspection viewer.

Blender material hierarchy and scene organization. The main *Qianqing Palace* views use corresponding principal-hall roof and terrace bounds; Figure 14a adds facades inspected through native orbit and zoom. Codex retains layered roofs, ridge ornaments, window divisions, a raised platform, and balustrades, with visible blue–green eave bands, although broad roof and wall surfaces appear pale. DSH clearly differentiates tile rows, painted eaves, and red door and window panels; its main facade has a less prominent projecting colonnade, and its side halls use simpler long roof and wall masses. CoworkOS separates golden roofs, red columns, lattice panels, painted eaves, and pale stonework into distinguishable components. The front colonnade, steps, and axial approach remain

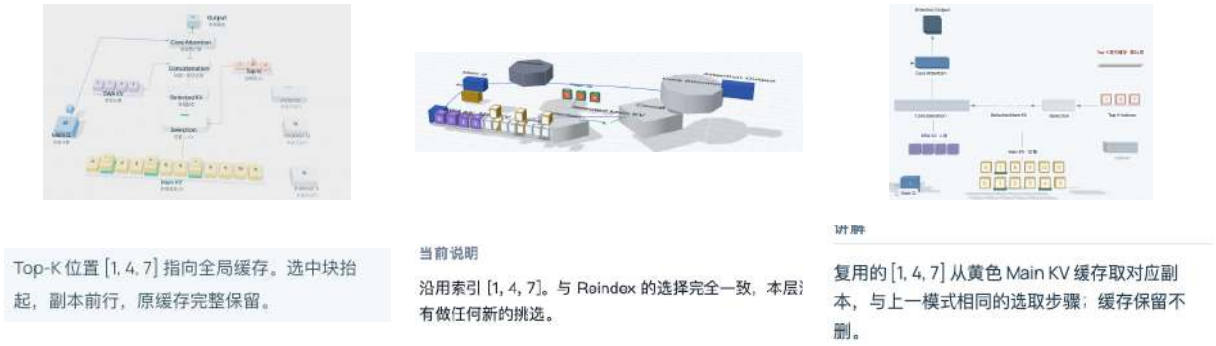
Codex DS Harness CoworkOS

(a) CREMA Espresso: expanded structure and component descriptions



Separated components with descriptions. Expanded view after native zoom-out. Expanded assembly with shorter introduction.

(b) CSA2 Attention: Reuse mode at the Selection stage



Reuses [1,4,7]; inactive indexing branch. Reuses [1,4,7]; retained selection explained. Reuses [1,4,7]; source and state explained.

Figure 12: Interactive visualization evidence. Columns show Codex, DSH, and CoworkOS. (a) Expanded CREMA Espresso assemblies and component-description crops; DSH uses its native zoom-out control. (b) CSA2 Attention in Reuse mode at Selection, with native explanation crops. English notes summarize the observations; native image content is unchanged.

identifiable after changing viewpoint. These observations concern delivered materials and architectural legibility, not historical accuracy; mesh counts are not treated as quality scores.

In the main *After the Fall* view, CoworkOS connects foreground vehicles, a mid-street bus, damaged multi-storey buildings, and distant shells into a continuous street. Cables span the road, and an emissive hotel sign anchors the right facade. Codex also has pronounced depth: its NO FUTURE and SECTOR 7 signs, dense rubble, and street equipment form a more crowded urban corridor. DSH combines a rusty vehicle, low-rise ruins, a bent street light, and an illuminated shelter in a more open setting. These differences concern spatial organization and asset composition, without establishing that CoworkOS leads on every scene detail. Alternate-view crops (b) retain vehicle and street details, including the visible gap between CoworkOS's bus wheels or lower body and the road. Spatial depth does not establish correct prop support or collision. Views retain delivered materials and presentation settings rather than calibrated shared lighting. The evaluation uses GLB exports, which contain no animation tracks; viewpoint motion does not establish model animation.

A.3 Automated Post-Training Run Records

Data contamination control. The initial seed data and all subsequent training data undergo BM25-based task-similarity filtering with a threshold of 0.8 to control near-duplicate overlap between train-

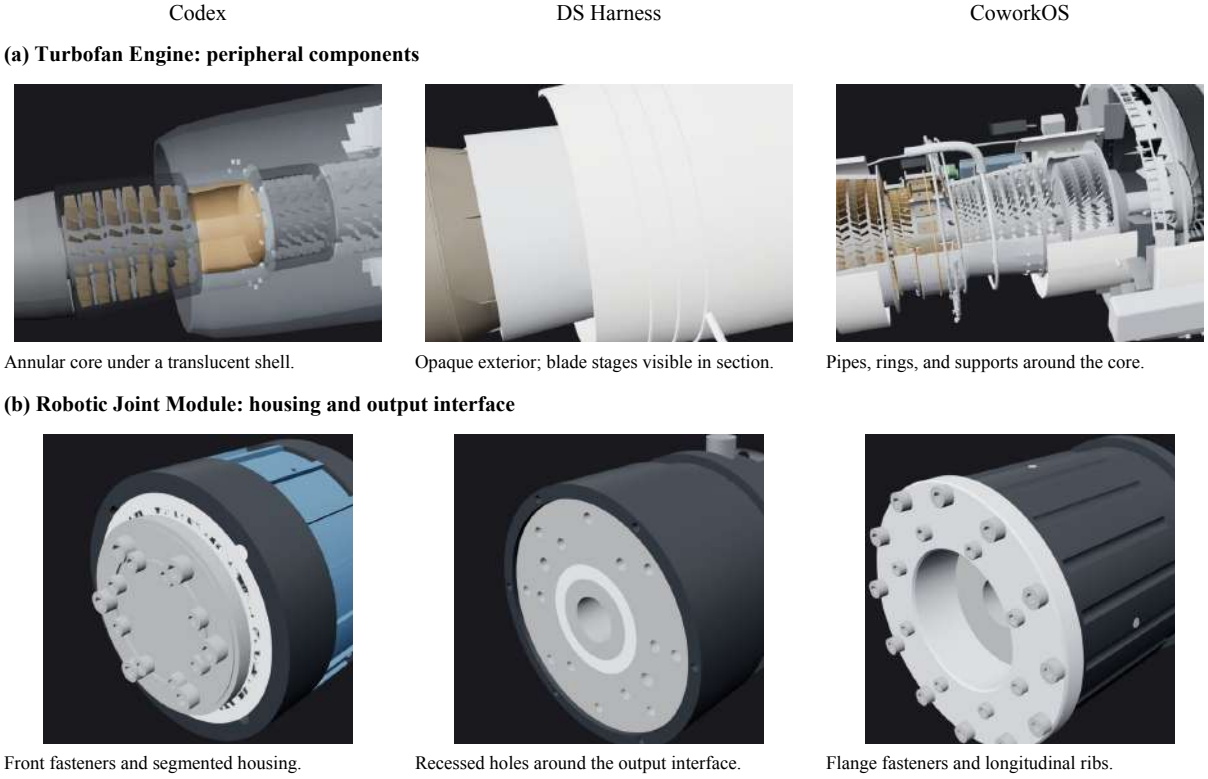


Figure 13: Additional CAD evidence. Columns show Codex, DSH, and CoworkOS. (a) Peripheral regions of assembled turbofan engines. (b) Robotic-joint housing and output interfaces. Each row uses the same inspection view and crop rule after axis alignment. Authored cutaways, transparency, and opaque surfaces are retained. Unretouched crops demonstrate visible structural detail; the shared half-sections in Figure 6e expose the internals.

ing and evaluation tasks. Training-history deduplication separately checks which data have already been used by the parent model.

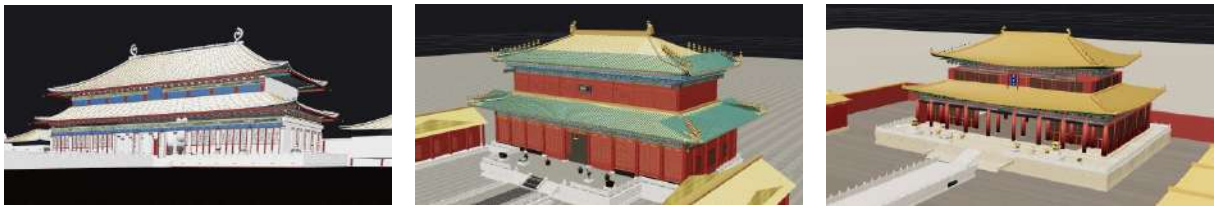
Scoring rules. All benchmarks follow their default scoring protocols. For GDPVal, the judge assigns partial credit according to task rubrics, and the reported score is $100 \sum_{i \in V} s_i / \sum_{i \in V} m_i$, where V is the set of validly judged tasks, s_i is the actual score, and m_i is the corresponding maximum. Actual scores and maxima are summed before division; this metric is neither Elo nor a reference win rate.

Complete results. Table 2 reports all eight benchmark scores and the overall average for each of the six rounds. Scores use a 0–100 scale; the aggregate is their arithmetic mean, with each benchmark contributing up to 12.5 points. Bold and underlining mark the highest and second-highest distinct scores per row, with identical markings for ties; the change column is excluded from ranking.

Training lineage and data. R0 is the baseline obtained through seed SFT from the base model. R1 also trains again from base, rather than adding another round of seed training to R0. From R2 onward, training-history checks select trajectories not previously used by the parent model, separately recording previously accepted but unused data and newly generated data. Independent trajectory counts differ from the number of sample presentations after repetition.

Codex DS Harness CoworkOS

(a) Qianqing Palace: facade and material hierarchy



Layered roof and eave bands; pale surfaces. Colored tiles and painted eaves over red walls. Golden roof, red columns, and pale stone base.

(b) After the Fall: vehicles and street dressing



Street equipment, rubble, and partial vehicle. Rusted vehicle and broken street surface. Bus wheels and lower body above the road.

Figure 14: Additional Blender scene evidence. Columns show Codex, DSH, and CoworkOS. (a) Corresponding main-hall facades after native orbit and zoom. (b) Vehicle and street-dressing crops from alternate views, retaining the visible bus–road gap in CoworkOS. Views retain the delivered models and presentation settings; they are not calibrated lighting comparisons. Only rectangular crops and proportional scaling are applied.

Benchmark	R0	R1	R2	R3	R4	R5	Δ R5–R0
Automation	11.00	35.50	30.83	39.49	41.83	**44.15**	+33.15
BFCL v4	63.48	61.81	61.76	62.11	61.76	**70.83**	+7.35
OfficeQA	55.64	53.38	52.63	56.14	59.40	**62.11**	+6.47
τ^3-Banking	14.43	19.59	16.49	18.57	**22.68**	**22.68**	+8.25
Harvey LAB	3.33	2.50	6.78	7.53	8.42	**11.08**	+7.75
SkillsBench	20.85	30.38	23.18	34.16	33.84	**35.76**	+14.91
GDPVal	74.11	75.43	**77.27**	75.14	76.45	76.45	+2.34
JobBench	**22.66**	16.93	18.27	20.58	21.32	20.53	−2.13
Average / 100	33.19	36.94	35.90	39.22	40.71	**42.95**	+9.76

Table 2: Complete results for AI-automated post-training across R0–R5 and the final change relative to the initial baseline.

Round	Start	Independent trajectories and composition	Epochs	Sampling multiplier	Steps
R0	base	1,345 seeds from five sources	1	1×	66
R1	base	1,345 seeds + 244 additions	1	1×	69
R2	R1	135 first-use: 77 previously retained + 58 newly generated	1	New 24× Old 1×	10
R3	R2	558 first-use: AB 437, OQ 85, τ^3-22, SB 14	4	New 24× Old 1×	24
R4	R3	453 first-use across AB, OQ, Tau3, SB	4	New 24× Old 1×	22
R5	R4	332 new: BFCL 203, AB 84, OQ 26, τ^3-17, JB 2	4	New 24× Old 1×	20

The five seed sources for R0 contain 320 open-source BC trajectories, 24 self-built BC trajectories, 261 DSQA trajectories, and two sleep-filtered WS sets of 627 and 113 trajectories.